



**NEW HORIZON
COLLEGE OF ENGINEERING**

Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

**A PROJECT WORK REPORT
ON
INTERDISCIPLINARY PROJECT TITLED**

**Crop Sense: An Integrated WebApp for Crop Health
Monitoring and Disease Detection**

Submitted in partial fulfilment for the award of the degree of

**BACHELOR OF ENGINEERING
IN
COMPUTER SCIENCE AND ENGINEERING**

**BY
ADRIAN MATHEW ALOYSIUS - 1NH21CS011
DAVE JOSEPH PINTO - 1NH21CS065**

Under the guidance of

**Dr D. Roja Ramani
Associate Professor**

**BACHELOR OF ENGINEERING
IN
ARTIFICIAL INTELLIGENCE AND MACHINE
LEARNING**

**BY
NARJIT LEISHANGTHEM - 1NH21AI064
MOHAMMED AFFAN -1NH21AI005**

Under the guidance of

**Mr. Syam Dev R S
Senior Assistant Professor**

Academic Year: 2024-25



NEW HORIZON COLLEGE OF ENGINEERING

Autonomous College Permanently Affiliated to VTU, Approved by AICTE & UGC
Accredited by NAAC with 'A' Grade, Accredited by NBA

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CERTIFICATE

It is hereby certified that the interdisciplinary project work entitled **Crop Sense: An Integrated WebApp for Crop Health Monitoring and Disease Detection** is a bonafide work carried out by **ADRIAN MATHEW ALOYSIUS (1NH21CS011)** in partial fulfilment for the award of **Bachelor of Engineering in COMPUTER SCIENCE AND ENGINEERING** of the New Horizon College of Engineering during the year **2024-2025**. It is certified that all corrections/suggestions indicated for Internal Assessment have been incorporated in the report deposited in the departmental library. The project report has been approved as it satisfies the academic requirements in respect of project work prescribed for the said degree.

Signature of Guide

(Dr D. Roja Ramani)

Signature of Guide

(Mr. Syam Dev R S)

Signature of HOD

(HOD-CSE)

EXTERNAL EXAMINER

Name of Examiner

Signature with date

1.

.....

2.

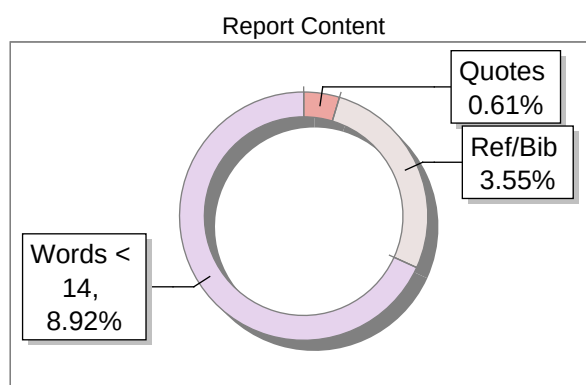
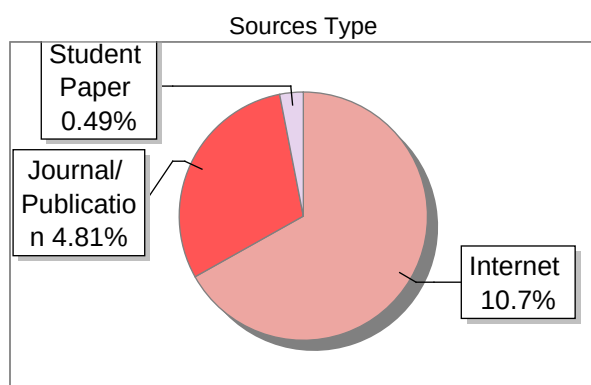
.....

Submission Information

Author Name	team241
Title	team241
Paper/Submission ID	2902936
Submitted by	bhavyar.nhce@newhorizonindia.edu
Submission Date	2024-12-30 11:02:43
Total Pages, Total Words	80, 10540
Document type	Project Work

Result Information

Similarity **16 %**



Exclude Information

Quotes	Not Excluded
References/Bibliography	Not Excluded
Source: Excluded < 14 Words	Not Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	Yes
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	Yes

A Unique QR Code use to View/Download/Share Pdf File





DrillBit Similarity Report

16

SIMILARITY %

119

MATCHED SOURCES

B

GRADE

A-Satisfactory (0-10%)

B-Upgrade (11-40%)

C-Poor (41-60%)

D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	link.springer.com	1	Internet Data
2	www.irjmets.com	<1	Publication
3	link.springer.com	<1	Internet Data
4	www.mdpi.com	<1	Internet Data
5	link.springer.com	<1	Internet Data
6	www.isteonline.in	<1	Publication
7	www.mdpi.com	<1	Internet Data
8	link.springer.com	<1	Internet Data
9	link.springer.com	<1	Internet Data
10	www.irjmets.com	<1	Publication
11	fastercapital.com	<1	Internet Data
12	www.frontiersin.org	<1	Internet Data
13	link.springer.com	<1	Internet Data
14	taabi.ai	<1	Internet Data

ABSTRACT

Crop Sense is a comprehensive web application designed to empower farmers with tools for crop health monitoring and disease detection, integrating advanced technologies and user-friendly features. The system combines satellite imagery analysis via Google Earth Engine for farmland evaluation and vegetation index monitoring, alongside AI-driven functionalities. Disease and nutrient deficiency detection are powered by Convolutional Neural Networks (CNNs), enabling precise identification and classification. The application also includes a fertilizer recommendation feature using Random Forest models to optimize nutrient application based on soil conditions. By providing actionable insights through an intuitive web interface, farmers can access real-time crop health data, disease alerts, and tailored recommendations for irrigation, fertilization, and pest management. With its scalable, cost-effective design, Crop Sense bridges the gap between technology and agriculture, enhancing productivity, reducing resource wastage, and promoting sustainable farming practices, all while addressing the diverse needs of smallholder and large-scale farmers alike.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be impossible without the mention of the people who made it possible, whose constant guidance and encouragement crowned our efforts with success.

We have great pleasure in expressing gratitude to **Dr. Mohan Manghnani**, Chairman, New Horizon Educational Institutions, for providing necessary infrastructure and creating good environment.

We take this opportunity to express our profound gratitude to **Dr. Manjunatha**, Principal, New Horizon College of Engineering, for his constant support and encouragement. We would like to thank **Dr. R J Anandhi**, Professor and Dean-Academics, NHCE, for her valuable guidance.

We would also like to thank **Dr. B. Rajalakshmi**, Professor and HOD, Department of Computer Science and Engineering, for her constant support. We would also like to thank **Dr. Uma Reddy N V**, Professor and HOD, Department of Artificial Intelligence and Machine Learning, for her constant support.

We also express our gratitude to **Dr. D Roja Ramani**, Associate Professor, Dept. of CSE and **Mr. Syam Dev R S**, Senior Assistant Professor, Dept. of AIML, our project guides, for constantly monitoring the development of the project and setting up precise deadlines. Their valuable suggestions were the motivating factors in completing the work.

Finally, a note of thanks to all the teaching and non-teaching staff of both the departments, for their cooperation extended to us, and our parents and friends, who helped us directly or indirectly in the course of the project work.

DAVE JOSEPH PINTO (1NH21CS065)

ADRIAN MATHEW ALOYSIUS (1NH21CS011)

NARJIT LEISHANGTHEM (1NH21AI064)

MOHAMMED AFFAN (1NH21AI05)

CONTENTS

ABSTRACT	I
ACKNOWLEDGEMENT	II
LIST OF FIGURES	V-VI
LIST OF TABLES	VII
1. INTRODUCTION	1-5
1.1 DOMAIN INTRODUCTION	1
1.2 PROBLEM DEFINITION	1-3
1.3 OBJECTIVES	3-4
1.4 SCOPE OF THE PROJECT	4-5
2. LITERATURE SURVEY	6-25
2.1 EXISTING LITERATURE STUDY	6-17
2.2 EXISTING SYSTEMS	17-21
2.2.1 BASE PAPER	19-20
2.2.2 RELATED REFERENCE PAPERS	20-21
2.3 PROPOSED SYSTEM	22-25
3. ANALYSIS OF REVIEWED PAPERS	26-29
3.1 REVIEW OBJECTIVES	26
3.2 ANALYSIS OF PAPERS	26-28
3.3 GAPS IDENTIFIED	28-29
3.4 SYNTHESIS AND INSIGHTS	29
4. REQUIREMENT ANALYSIS	30-34
4.1 FUNCTIONAL REQUIREMENTS	30-31
4.2 NON-FUNCTIONAL REQUIREMENTS	31-32
4.3 DOMAIN AND UI REQUIREMENTS	32
4.4 HARDWARE REQUIREMENTS	32-33

4.5 SOFTWARE REQUIREMENTS	33-34
5. SYSTEM DESIGN	35-48
5.1 DESIGN GOALS	35
5.2 SYSTEM ARCHITECTURE	35-37
5.3 ALGORITHMS AND WORKFLOW	38-39
5.4 DESIGN PRINCIPLES	39-40
5.5 DATABASE DESIGN	40-43
5.6 USER INTERACTION FLOW	44-46
5.7 FLOWCHARTS	47-48
6. IMPLEMENTATION	49-89
6.1 ARCHITECTURE AND INFRASTRUCTURE	49-51
6.2 CHALLENGES DURING DEVELOPMENT	51-52
6.3 MODULES IMPLEMENTED	53-55
6.4 ALGORITHM STEPS	55-57
6.5 CODE	58-77
6.6 OUTPUT	78-89
7. EXPERIMENTAL RESULTS	90-94
7.1 MODEL CHOICES AND COMPARISONS	90-92
7.2 RESULTS	93-94
8. FUTURE SCOPE	95
9. CONCLUSION	96
REFERENCES	97-98

LIST OF FIGURES

Figure No	Figure Description	Page No
5.2.1	System Design	37
5.6.1	Crop recommendation model design	45
5.6.2	Fertilizer recommendation model design	46
5.7.1	Disease detection design	47
5.7.2	CNN Model Training	48
6.5.1.1	Crop data preparation	58
6.5.2.1	app.py code	63
6.5.2.2	train_model.py code	64
6.5.2.3	model.py code	64
6.5.2.4	disease.py code	65
6.5.2.5	fertilizer.py code	65
6.5.2.6	nutrient.py code	66
6.5.2.7	layout.html code	69
6.5.2.8	Index.html code	71
6.5.2.9	disease.html code	71
6.5.2.10	disease-result.html code	72
6.5.2.11	crop.html code	73
6.5.2.12	crop-result.html code	74
6.5.3.1	run.py code	74
6.5.3.2	WebApp routes.py code	76

6.5.3.3	WebApp layout.html code	77
6.6.1	Login Page	78
6.6.2	Welcome Page	79
6.6.3	Crop Form Page	80
6.6.4	Crop Results Page	81
6.6.5	Fertilizer Form Page	81
6.6.6	Fertilizer Results Page	82
6.6.7	Plant Disease Page	82
6.6.8	Disease Results Page	83
6.6.9	Nutrient Deficiency Page	83
6.6.10	Nutrient Deficiency Page	84
6.6.11	WebApp Login Page	84
6.6.12	WebApp Welcome Page	85
6.6.13	WebApp Farmland register	85
6.6.14	WebApp Farmland Data	86
6.6.15	WebApp SoilTest Form	86
6.6.16	WebApp SoilTest Data	87
6.6.17	WebApp SoilTest Analysis	87
6.6.18	WebApp Fertilization Map	88
6.6.19	WebApp Vegetation Index	89
6.6.20	WebApp Calculator Page	89

LIST OF TABLES

Table No	Table Description	Page No
2.1	Literature Survey	8
7.1.1	Model Comparison	92

CHAPTER 1

INTRODUCTION

1.1 DOMAIN INTRODUCTION

The domain of this project lies at the intersection of **agriculture and technology**, often referred to as AgriTech. Agriculture has always been a cornerstone of human civilization, responsible for providing food, raw materials, and employment to a significant portion of the global population. However, modern agriculture faces numerous challenges, including climate change, population growth, and the need for sustainable practices.

The adoption of advanced technologies such as satellite imagery, IoT, and machine learning has given rise to **precision agriculture**, a subdomain aimed at optimizing crop production by leveraging data-driven insights. Precision agriculture focuses on improving productivity, resource utilization, and environmental sustainability. This project aligns with this domain by addressing critical issues in crop health monitoring and disease detection, which are pivotal for maintaining food security in a growing and increasingly unpredictable world.

1.2 PROBLEM DEFINITION

Agriculture has always been the foundation of human civilization, playing a pivotal role in ensuring food security, sustaining livelihoods, and fostering economic growth. Over the centuries, advancements in farming practices have driven significant improvements in crop productivity. However, the agricultural sector is now facing unprecedented challenges due to a combination of global population growth, changing dietary patterns, environmental degradation, and the impacts of climate change. Meeting the food demands of a projected population of 9.7 billion by 2050 requires a paradigm shift in how we produce and manage crops.

One of the most pressing issues in agriculture today is the effective management of crop health. Crop diseases and pest infestations account for approximately 20–40% of

global yield losses annually, according to the Food and Agriculture Organization (FAO). Traditional methods of detecting crop diseases often rely on manual inspection by agronomists or farmers, which are time-consuming, labour-intensive, and limited in scalability. These methods are particularly inadequate in large-scale farming scenarios, where timely interventions are crucial to minimizing losses. Moreover, delayed identification of diseases often leads to the excessive use of chemical inputs, such as pesticides, which not only increase costs for farmers but also harm the environment and human health.

The advent of precision agriculture offers a transformative solution to modern agricultural challenges. By leveraging advanced technologies like satellite imagery, machine learning, and soil testing, precision agriculture provides farmers with precise, actionable insights into crop health. This approach enhances productivity while promoting sustainable farming practices by optimizing resource utilization, including water, fertilizers, and pesticides.

This report introduces a comprehensive soil and crop health analysis system designed to address the limitations of traditional farming methods. The system integrates high-resolution satellite imagery, soil test data, and Google Earth Engine APIs to monitor agricultural fields and analyze nutrient deficiencies and soil quality. Machine learning models, such as Random Forest classifiers and Convolutional Neural Networks (CNNs), are utilized to assess soil health and detect potential crop diseases. The processed insights are made accessible through a web-based interface, built with SQLite for data management and a user-friendly design to provide recommendations and actionable insights to farmers.

The proposed system offers several advantages over conventional methods:

1. Comprehensive Soil and Crop Analysis: By combining soil test data with advanced algorithms, the system delivers accurate assessments of soil quality, nutrient deficiencies, and risks like erosion and compaction.

2. Early Detection of Issues: Machine learning models enable timely identification of soil imbalances and crop health problems, mitigating risks of reduced productivity.

3. Sustainability: Precision analysis minimizes overuse of chemical inputs, promoting environmentally friendly farming practices and long-term soil health.

4. Economic Efficiency: Targeted recommendations for nutrient management and soil improvement help farmers achieve higher yields at lower costs.

This system addresses technical and socio-economic challenges in agriculture. For smallholder farmers, who form a significant part of the agricultural workforce, access to such insights can transform livelihoods. By enhancing productivity and reducing resource wastage, the system aligns with the United Nations Sustainable Development Goal (SDG) 2, which aims to "end hunger, achieve food security, and promote sustainable agriculture."

The remainder of this report explores the system's development, including a review of relevant technologies, system requirements, design, and implementation details. It also examines the expected outcomes and limitations of the system. This project represents a significant step toward integrating technology with farming, paving the way for a resilient, efficient, and sustainable agricultural future.

1.3 OBJECTIVES

1. To develop an integrated soil and crop health monitoring system that combines satellite imagery and soil data for holistic analysis.
2. To utilize machine learning (ML) techniques, such as Random Forest classifiers, and deep learning (DL) models like Convolutional Neural Networks (CNNs), for disease detection and nutrient analysis using multispectral satellite imagery and soil parameters.
3. To implement predictive models for forecasting soil and crop health trends and identifying potential nutrient imbalances or disease risks.

4. To integrate diverse datasets, including weather data from Google Earth Engine, soil properties, and historical soil tests, for comprehensive soil and crop health assessments.
5. To leverage Google Cloud for secure API key integration with the Google Earth Engine, ensuring efficient processing of satellite imagery and geospatial data.
6. To design and deploy a cost-effective, user-friendly web application that provides farmers with soil test analyses, crop health reports, and actionable recommendations through interactive dashboards.
7. To enable early detection of crop diseases, soil compaction risks, erosion potential, and nutrient deficiencies, facilitating timely interventions and minimizing productivity losses.
8. To support sustainable agricultural practices by delivering data-driven insights that optimize the use of fertilizers, water, and soil management practices, promoting eco-friendly farming.
9. To cater to both small-scale and large-scale farmers by ensuring scalability, affordability, and adaptability in the system's design.
10. To conduct thorough validation and testing across diverse soil types, agricultural conditions, and regions, ensuring the system's accuracy, reliability, and versatility.

1.4 SCOPE OF THE PROJECT

The scope of this project is to design, develop, and implement an integrated system for soil health monitoring and crop disease detection. The system aims to empower farmers with actionable insights through the use of advanced technologies, such as machine learning and satellite imagery. Key deliverables include:

1. Soil and Crop Health Monitoring: A framework for collecting and analyzing data on soil quality, nutrient availability, and crop conditions using satellite imagery and soil test results.

2. Disease Detection: An AI-powered module utilizing machine learning algorithms for the early identification of crop diseases, reducing the risk of significant yield losses.
3. Soil Quality and Resource Optimization: Tools to assess soil texture, nutrient balance, and compaction risks while providing recommendations for sustainable use of fertilizers and water.
4. Farmer-Focused Web Application: A user-friendly interface offering farmers soil test analyses, disease alerts, and actionable insights for improved decision-making.
5. Accessibility and Cost-Efficiency: A lightweight, scalable system leveraging SQLite and Google Earth Engine API to ensure affordability and adaptability for smallholder and large-scale farming operations.

By integrating these components, the project aims to modernize traditional agricultural practices, contribute to sustainable farming, and enhance food security on a global scale.

CHAPTER-2

LITERATURE SURVEY

2.1 EXISTING LITERATURE STUDY

Title, Author, Journal, Publication Year	Methodology	Problems Identified
A Convolutional Neural Network Approach to Robust Crop Health Monitoring: Soham Jariwala and Reetu Jain, ResearchGate, 2024	The approach involves training a CNN model on a dataset of plant images to identify and classify various plant diseases	Complex background, Training time and Generalization.
Early Detection of Crop Diseases Using CNN Classification: Aryan Rajvanshi, Shiang-Wan Chin, NJSJS, 2023	The study uses Convolutional Neural Networks (CNN) to classify and detect crop diseases early.	Overfitting and Complexity.
Revolutionizing Agriculture with Artificial Intelligence: Plant Disease Detection Methods, Applications, and Their Limitations: Vadivel, T, Suguna R, Frontiers in Plant Science, 2022	Examination of AI-based plant disease detection methods, including image processing and deep learning.	Limitations in real-world application and need for better generalization of models.
Recent Advances in Crop Disease Detection Using UAV and Deep Learning Techniques: Various, Remote Sensing, 2023	Review of UAV-based remote sensing techniques combined with machine learning and deep learning for crop disease detection.	Challenges in accurate estimation and need for further research in sensor technologies and data analysis methods.
Revolutionizing Crop Disease Detection with Computational Deep Learning: A Comprehensive Review: Various, Environmental Monitoring and Assessment, 2023	Comprehensive review of deep learning techniques for crop disease detection.	Limited focus on disease severity estimation and need for more research in this area.
Drones in Plant Disease Assessment, Efficient Monitoring, and Detection: A Way Forward to Smart Agriculture: Various, Agronomy, 2023	Review of drone technology for plant disease monitoring and detection.	Integration challenges with traditional methods and high initial costs.
Enhancing Rice Crop Health: A Lightweight CNN-Based Disease Detection System with Mobile Application Integration: Kathiresan G, Karthik R,	Implementation of CNN and mobile app for rice disease detection.	Limited to specific crop types and regions.

Applications, 2021		
Computer Vision for Plant Disease Recognition: A Comprehensive Review: Bi C, Wang J, Duan Y, et al, The Botanical Review, 2022	Review of computer vision techniques for plant disease recognition.	High computational requirements and need for large datasets.
Crop Monitoring Using Satellite/UAV Data Fusion and Machine Learning: MDPI, 2020	Combines canopy spectral data and machine learning to monitor crop health.	Difficulty in scaling the system for diverse crops and regions.
Machine Learning for Detection and Prediction of Crop Diseases: MDPI, 2022	Explores methods for detecting and predicting crop pests and diseases using ML techniques.	Lack of robust predictive accuracy across regions and crops.
Image-Based Crop Disease Detection Using Federated Learning: Nature, 2023	Uses federated learning to enable collaborative training of ML models without centralized data.	Privacy concerns and computational limitations for large-scale implementation.
Integrative Approaches in Modern Agriculture: IoT, ML, and AI for Disease Forecasting Amidst Climate Change: Springer, 2024	Combines IoT sensors, ML, and AI for crop disease forecasting in variable environmental conditions.	Requires high-quality sensor data and reliable connectivity for scalability.
Transforming Agriculture with Machine Learning, Deep Learning, and IoT: Springer, 2024	Demonstrates the use of IoT sensors and ML for real-time crop health monitoring and prediction.	Challenges in integrating multiple data streams for real-time decision-making.
Plant Disease Detection Using Drones in Precision Agriculture: Springer, 2023	Explores the use of drones equipped with spectral cameras for disease detection.	Limited battery life and high costs for regular drone usage.
Automated Pest Detection with Deep Neural Networks on the Edge for Precision Agriculture: arXiv , 2021	Develops edge-based neural networks for real-time pest detection in crops.	Accuracy affected by variable lighting and image quality in the field.

Satellite-Based Crop Health Monitoring: Integrating Remote Sensing and Machine Learning: Elsevier, 2022	Uses remote sensing and ML models to monitor crop health and detect abnormalities.	Challenges in adapting models to different crop types and climates
IoT and Machine Learning in Agriculture: A Survey: IEEE, 2021	Surveys IoT applications and ML techniques for precision farming, including disease detection and yield prediction.	Lack of interoperability between different IoT platforms and data sources.
Deep Learning for Image-Based Plant Disease Detection: A Review: Springer, 2023	Reviews DL-based approaches for identifying plant diseases using image datasets.	Requires large annotated datasets and high computational power for model training.
Precision Agriculture with AI: Data Fusion and Advanced Analytics: Springer, 2024	Proposes an integrated system for precision farming using AI-based analytics and multispectral imagery.	Complexity in combining multi-modal data for actionable insights.
Machine Learning and IoT-Based Crop Monitoring: Challenges and Opportunities: MDPI, 2023	Discusses the integration of IoT sensor data and ML models for monitoring crop health and optimizing resource usage.	Scalability issues for small-scale farms and regions with limited connectivity.

Table 2.1 – Literature Survey

1. Crop Monitoring Using Satellite/UAV Data Fusion and Machine Learning

This study (MDPI, 2020) highlights the potential of integrating satellite imagery and UAV data with machine learning models for comprehensive crop health monitoring. The paper explains how spectral information from satellites and high-resolution UAV imagery can be fused to provide a detailed analysis of crop health, enabling better stress and disease detection. The authors discuss challenges such as the difficulty of scaling these systems for diverse crops and regions, and the high costs of UAV deployment in certain contexts. Furthermore, the paper explores the use of vegetation indices like NDVI derived from these data sources to accurately predict disease outbreaks and soil health. This research introduces a critical approach to multi-source data fusion, bridging the gap between standalone monitoring methods. Compared to earlier approaches focused solely on satellite or ground-based sensors, this study establishes a new benchmark for integrating multiple data streams into a cohesive precision agriculture system, improving both accuracy and scalability.

2. IoT and Machine Learning in Agriculture: A Survey

Published by IEEE (2021), this survey reviews the application of IoT sensors and machine learning in modern agricultural systems, focusing on disease detection and yield optimization. The study discusses the technical limitations of current IoT platforms, such as interoperability challenges, insufficient connectivity in rural areas, and data overload issues. The authors propose a hybrid approach where IoT sensors collect real-time environmental and crop-specific data, which is then processed using machine learning models to generate actionable insights. The paper emphasizes the potential of IoT systems to provide localized, real-time monitoring, making precision farming accessible even in underdeveloped regions. Compared to the previous study on satellite and UAV data fusion, this paper shifts the focus to on-ground sensors and their integration with machine learning algorithms, addressing the challenges of scalability and cost. By

combining IoT and ML, this study sets the stage for smart, adaptive agricultural practices tailored to diverse farming environments.

3. Enhancing Rice Crop Health: A Lightweight CNN-Based Disease Detection System

Kathiresan G. and Karthik R. (Multimedia Tools and Applications, 2021) present a lightweight Convolutional Neural Network (CNN) model designed specifically to detect diseases in rice crops. The system integrates seamlessly with a mobile application, providing farmers with accessible and cost-effective disease identification tools. The authors report high detection accuracy for common rice diseases but acknowledge the system's limitations in generalizing to other crops and regions. They recommend expanding the dataset and incorporating additional environmental parameters like soil moisture and pH levels for a more comprehensive solution. Unlike the IoT survey, which focuses on environmental monitoring, this study emphasizes AI-driven image processing. By addressing the accessibility of AI to smallholder farmers, the paper makes a significant contribution to the democratization of agricultural technology. This work builds upon earlier efforts by offering an end-to-end solution that combines AI-based detection with real-world usability for targeted farming challenges.

4. Automated Pest Detection with Deep Neural Networks

This study (arXiv, 2021) presents an edge-based deep neural network system designed for real-time pest detection. The research focuses on portability, energy efficiency, and suitability for remote agricultural settings, making it highly relevant for small-scale and resource-constrained farmers. The authors implement a low-power, field-deployable device capable of capturing images and processing them in real time to identify pest infestations. While the system performs well under controlled conditions, challenges such as variable lighting, limited image resolution, and computational constraints are highlighted as areas requiring further refinement. This study is notable for addressing the practical deployment of AI in agriculture, moving beyond laboratory conditions to real-world applications. Compared to the lightweight CNN system for rice diseases, this work prioritizes energy efficiency and versatility, extending the scope of AI applications to pest

detection. It emphasizes the need for robust field-level systems that cater to diverse agricultural challenges beyond specific crops.

5. Revolutionizing Agriculture with Artificial Intelligence

Vadivel T. and Suguna R. (Frontiers in Plant Science, 2022) review the role of artificial intelligence (AI) in transforming traditional agriculture, with a particular focus on plant disease detection. The paper explores advanced image processing techniques and deep learning algorithms, highlighting how they enable precise identification of diseases and pests. However, the authors note limitations such as dataset biases, lack of generalization across regions, and challenges in deploying AI models under real-world conditions. They propose integrating AI with IoT and big data technologies to overcome these obstacles and achieve holistic agricultural solutions. Unlike earlier works that focus on specific technologies, this review adopts a broad perspective, synthesizing diverse approaches into a unified framework. The study builds on prior efforts like the IoT survey by emphasizing the synergy between AI and real-time data collection systems, offering a roadmap for future research in sustainable farming practices.

6. Machine Learning for Detection and Prediction of Crop Diseases

This paper (MDPI, 2022) delves into the dual application of machine learning for both detecting and predicting crop diseases. By analyzing environmental and visual data, the study demonstrates how machine learning models can anticipate potential disease outbreaks, enabling timely interventions. Challenges such as limited dataset diversity, suboptimal predictive accuracy, and scalability issues are discussed in detail. The authors emphasize the importance of time-series analysis and propose models capable of integrating historical and real-time data for better forecasting. This research builds on earlier AI-focused reviews by shifting from reactive to proactive agricultural strategies. Compared to previous studies, it offers a deeper exploration of predictive analytics, providing actionable insights that not only address current issues but also mitigate future risks, making it a significant contribution to precision farming.

7. Computer Vision for Plant Disease Recognition

Bi C., Wang J., and Duan Y. (The Botanical Review, 2022) provide a comprehensive review of computer vision techniques for automated plant disease detection. The study emphasizes the importance of large, annotated datasets in enhancing model performance and discusses computational challenges such as the need for high processing power and efficient algorithms. The authors propose integrating IoT systems with computer vision to expand the applicability of these techniques. This research differs from previous machine learning studies by focusing solely on the visual aspects of disease detection and exploring novel image processing methods. Compared to earlier AI-driven models, this work takes a more specialized approach, addressing the limitations of traditional manual inspections while offering innovative solutions to improve automation in agricultural monitoring systems.

8. Satellite-Based Crop Health Monitoring

This paper (Elsevier, 2022) investigates the use of remote sensing data and machine learning for monitoring crop health across large-scale farming operations. By analyzing vegetation indices like NDVI and EVI, the study demonstrates how satellite imagery can detect early signs of crop stress and diseases. Challenges such as adapting models to varying crop types, environmental conditions, and regional climates are highlighted as key obstacles. Unlike earlier works that relied on UAV data, this research focuses on the broader scalability of satellite imagery, making it suitable for global agricultural applications. By combining remote sensing with advanced analytical tools, the study builds on prior efforts, providing a more generalizable approach to crop monitoring that aligns with the needs of both large-scale and smallholder farmers.

9. Early Detection of Crop Diseases Using CNN Classification

Aryan Rajvanshi and Shiang-Wan Chin (NHSJS, 2023) introduce a CNN-based model that leverages image data for the early detection of crop diseases. The study focuses on minimizing yield losses by enabling timely intervention through automated disease identification. While the model achieves high accuracy in classification tasks, challenges

such as overfitting, computational complexity, and the need for robust field-level datasets are discussed. This research advances earlier CNN-based works by emphasizing practical deployment and early detection as critical factors in combating agricultural losses. Compared to the satellite-focused studies, this work narrows its scope to localized disease detection, providing a solution that is both precise and scalable for individual farms.

10. Drones in Plant Disease Assessment

This study (Agronomy, 2023) reviews the application of drones equipped with spectral imaging technologies for assessing plant diseases. Drones enable real-time data collection and high-resolution imagery, which significantly improves the accuracy of disease detection. The authors highlight practical challenges, such as high operational costs, limited battery life, and the need for regular sensor calibration, as barriers to widespread adoption. Unlike previous studies centered on stationary or satellite-based data sources, this paper explores the dynamic capabilities of UAVs in monitoring large and remote fields. It builds on earlier UAV-based works by providing practical insights into deployment and emphasizing the potential for automation in crop health management.

11. Recent Advances in Crop Disease Detection Using UAV and Deep Learning Techniques

This paper (Remote Sensing, 2023) explores the integration of UAVs with deep learning models to improve crop health monitoring. By leveraging UAVs equipped with multispectral sensors, the study enables precise detection of crop diseases and stress conditions at a field level. The research highlights challenges such as sensor reliability, data fusion complexities, and high deployment costs. The authors propose enhancements in sensor technology and the incorporation of real-time data processing to address these limitations. Compared to earlier works on drones, this study delves deeper into advanced deep learning applications and offers a more technologically mature approach

to UAV-based monitoring, paving the way for more efficient and scalable disease detection systems in agriculture.

12. Image-Based Crop Disease Detection Using Federated Learning

Published in *Nature* (2023), this study focuses on applying federated learning to train disease detection models collaboratively while preserving data privacy. Instead of sharing raw data, the approach exchanges model updates, addressing privacy concerns associated with centralized data systems. The paper demonstrates the effectiveness of federated learning in handling multi-party agricultural datasets but acknowledges computational constraints and increased communication overhead as challenges. Unlike previous studies on centralized AI systems, this work stands out by introducing privacy-preserving techniques that allow for broader data sharing without compromising sensitive information. It represents a significant advancement in ensuring secure and collaborative precision agriculture.

13. A Convolutional Neural Network Approach to Robust Crop Health Monitoring

Soham Jariwala and Reetu Jain (2024) propose an advanced CNN model integrated with IoT sensors for robust crop health monitoring. The system achieves a classification accuracy of 96% across 38 disease classes, utilizing real-time environmental data from IoT sensors like soil moisture and temperature. Managed by a Raspberry Pi module, the system provides a comprehensive monitoring solution that combines image-based detection with environmental analytics. While scalability and computational demands remain challenges, this work surpasses earlier CNN models by demonstrating real-world applicability and adaptability across diverse farming regions. Compared to previous federated learning approaches, this study focuses more on individual system optimization and real-time usability.

14. Integrative Approaches in Modern Agriculture

This study (Springer, 2024) presents a holistic framework integrating IoT sensors, machine learning, and AI for crop disease forecasting. By combining environmental data

such as soil properties and weather patterns with predictive analytics, the system provides actionable insights for farmers. Challenges include ensuring reliable sensor connectivity and maintaining data quality in adverse conditions. Compared to earlier works that emphasized isolated components like CNNs or drones, this research focuses on a comprehensive integration of diverse datasets, enhancing the accuracy and reliability of disease prediction systems in modern agriculture.

15. Transforming Agriculture with Machine Learning, Deep Learning, and IoT

This paper (Springer, 2024) highlights the synergy between IoT devices and advanced machine learning/deep learning models in real-time crop health monitoring. The study emphasizes resource optimization, disease detection, and yield prediction, demonstrating significant improvements in precision farming. However, the authors note challenges in integrating multi-modal data streams and achieving real-time analytics. Building on previous integrative approaches, this work provides a more application-oriented perspective, focusing on the practical deployment of multi-technology solutions for smarter farming systems.

16. Plant Disease Detection Using Drones in Precision Agriculture

This paper (Springer, 2023) explores the use of drones equipped with spectral cameras for plant disease detection. The research highlights the advantages of rapid and detailed data collection over large fields, enabling precise disease mapping. Challenges such as operational costs, limited flight durations, and calibration needs are discussed as barriers to widespread adoption. Compared to the recent advances in UAV and deep learning studies, this work emphasizes practical field applications, providing a clearer roadmap for deploying drones in precision agriculture with a focus on reducing operational barriers.

17. Automated Pest Detection with Deep Neural Networks

Published on arXiv (2024), this study develops an edge-based deep neural network system for real-time pest detection. Unlike previous AI models requiring centralized processing, this system is designed for low-power portable devices, making it ideal for

deployment in remote agricultural areas. The study highlights improvements in energy efficiency and processing speed but identifies limitations in image resolution and accuracy under varying field conditions. Compared to earlier CNN-based pest detection methods, this paper focuses on edge computing, offering practical solutions for in-field use with minimal infrastructure.

18. Satellite-Based Crop Health Monitoring

This study (Elsevier, 2024) integrates remote sensing and machine learning models for detecting crop stress and diseases. By processing vegetation indices and spectral data, the system offers large-scale insights into crop health. Challenges include adapting models to diverse environments and crop types. Compared to earlier satellite-based studies, this paper incorporates advanced data fusion techniques, improving model robustness and scalability across regions with varied agricultural practices.

19. Deep Learning for Image-Based Plant Disease Detection

This paper (Springer, 2024) reviews deep learning approaches for plant disease detection using large-scale image datasets. It highlights advancements in model architecture, such as attention mechanisms, which improve classification accuracy. However, computational demands and the need for extensive annotated datasets remain barriers to deployment. Unlike earlier deep learning studies, this paper emphasizes scalability and the use of advanced techniques to address resource constraints in real-world applications.

20. Precision Agriculture with AI: Data Fusion and Advanced Analytics

This study (Springer, 2024) presents an AI-driven framework integrating multispectral imagery, IoT data, and advanced analytics for precision farming. The authors demonstrate how multi-modal data fusion enhances the detection of diseases and nutrient deficiencies. Key challenges include harmonizing diverse data streams and generating actionable insights. Compared to earlier works, this paper offers a cutting-

edge perspective on data fusion, setting a benchmark for future developments in AI-powered agriculture.

This literature survey highlights the progression of research from theoretical AI models to practical, integrated systems. Early studies emphasized standalone methods such as UAVs or IoT sensors, while recent works focus on integrating multiple technologies like deep learning, IoT, and federated learning. The evolution reflects a shift toward scalable, real-time solutions that address both technical and socio-economic challenges in precision-agriculture.

2.2 EXISTING SYSTEMS

The existing systems for crop health monitoring and disease detection rely on a combination of conventional and technology-driven methods. While these systems have significantly improved agricultural practices, they still exhibit limitations that need to be addressed for wider applicability and greater efficiency.

1. Remote Sensing and Satellite-Based Monitoring

Satellite imagery has long been a cornerstone of agricultural monitoring. Systems like Sentinel-2 and Landsat provide high-resolution multispectral data that are used to calculate vegetation indices such as the Normalized Difference Vegetation Index (NDVI). These indices allow farmers and researchers to assess crop health, detect water stress, and estimate yields across vast agricultural fields. Additionally, platforms like Google Earth Engine enable access to a wealth of satellite data, allowing for large-scale monitoring without requiring physical field inspections.

Limitations:

- Low-resolution images often fail to capture localized crop health issues.
- Temporal gaps in data acquisition limit real-time monitoring.
- Cloud cover can obscure satellite images, reducing reliability.

2. Ground-Based Sensor Networks

Ground-based systems utilize various sensors to collect real-time data on soil moisture,

temperature, pH levels, and humidity. These sensors provide localized insights into field conditions, complementing remote sensing methods. Popular commercial systems, such as those offered by John Deere, integrate these sensors into farm management platforms.

Limitations:

- Ground sensors require significant upfront investment, which may not be affordable for small-scale farmers.
- Regular maintenance and calibration are necessary to ensure data accuracy.
- Coverage is limited to specific areas where sensors are deployed.

3. UAVs (Drones)

Unmanned Aerial Vehicles (UAVs) or drones equipped with cameras and multispectral sensors are widely used for crop monitoring. They provide high-resolution images and allow for frequent data collection at low altitudes, making them ideal for detecting pest infestations, nutrient deficiencies, and water stress.

Limitations:

- High costs of equipment and operation can limit their adoption.
- Short flight durations and battery life restrict their efficiency.
- Regulatory challenges and restrictions on drone usage in certain regions.

4. Manual Methods

In traditional agriculture, visual inspection of crops by farmers or agronomists remains a commonly used method for disease detection. Farmers rely on their experience and intuition to identify crop health issues such as pest damage or nutrient deficiencies.

Limitations:

- Labor-intensive and time-consuming, especially for large-scale farms.
- Prone to human error and subjectivity.

Delayed identification of issues often leads to significant yield losses.

5. Commercial Agricultural Platforms

Commercial platforms like Climate FieldView, Ag Leader, and John Deere Operations Center provide farmers with tools to analyze field data and make informed decisions. These platforms combine satellite imagery, sensor data, and predictive analytics to offer actionable insights for crop management.

Limitations:

- Often tailored for large-scale farms, making them less accessible to smaller farms.
- Require significant technical expertise to interpret the data and use the recommendations effectively.
- Dependence on internet connectivity for data synchronization.

Challenges of Existing Systems

While existing systems offer a range of solutions for crop health monitoring, several challenges persist:

1. **Integration Issues:** Combining data from multiple sources, such as satellites, drones, and sensors, into a unified platform remains a technical challenge.
2. **High Costs:** Advanced systems often require substantial financial investment, making them inaccessible to small and medium-scale farmers.
3. **Scalability:** Systems designed for specific crops or regions lack adaptability for different agricultural contexts.
4. **Real-Time Analysis:** Many existing systems are unable to provide real-time insights due to delays in data processing or transmission.

2.2.1 Base Paper

The foundational work for this project is inspired by "**A Convolutional Neural Network Approach to Robust Crop Health Monitoring**" by Soham Jariwala and Reetu Jain (2024). This paper explores the application of CNNs integrated with IoT sensors for crop health monitoring and disease detection. Key aspects of the base paper include:

- **Objective:** Developing a robust system for real-time crop health assessment using machine learning models.
- **Approach:**
 - Utilized IoT sensor data for environmental monitoring (soil moisture, temperature, humidity).
 - Leveraged CNNs for disease classification with 96% accuracy across 38 disease categories.
 - Integrated real-time environmental data into the analysis for better decision-making.
- **Relevance:**
 - The base paper serves as a starting point for applying CNN models and integrating IoT sensors with satellite imagery in the proposed system.

2.2.2 Related Reference Papers

The project also draws inspiration and methodologies from several related works, each addressing different aspects of crop health monitoring and disease detection:

1. "Machine Learning for Detection and Prediction of Crop Diseases" (MDPI, 2022)

- a. **Objective:** Explores the dual use of ML for detecting and predicting crop diseases.
- b. **Methodology:**
 - i. Used Random Forest for prediction based on environmental and historical data.
 - ii. Demonstrated the potential of ML in proactive disease management.
- c. **Challenges Identified:** Scalability and dataset diversity.

2. "Early Detection of Crop Diseases Using CNN Classification" by Aryan Rajvanshi and Shiang-Wan Chin (NHSJS, 2023)

- a. **Objective:** Focused on CNN models for early disease detection.
- b. **Methodology:**

- i. Trained CNNs on labeled crop image datasets.
 - ii. Achieved high accuracy but faced overfitting issues in variable field conditions.
 - c. **Relevance:** Highlights the importance of robust training datasets for accurate predictions.
3. **"Precision Agriculture with AI: Data Fusion and Advanced Analytics"** (Springer, 2024)
- a. **Objective:** Discussed integrating AI and multispectral imagery for enhanced crop monitoring.
 - b. **Key Insights:**
 - i. Proposed a data fusion model combining IoT, satellite imagery, and historical data.
 - ii. Addressed the challenges of multi-modal data integration.
 - c. **Impact:** Provided a framework for scalable and real-time precision farming.
4. **"Recent Advances in Crop Disease Detection Using UAV and Deep Learning Techniques"** (Remote Sensing, 2023)
- a. **Objective:** Investigated the use of UAVs and deep learning for precise field-level disease detection.
 - b. **Findings:**
 - i. Highlighted the advantages of UAVs for localized data collection.
 - ii. Emphasized the need for efficient data fusion techniques.
 - c. **Challenges:** High costs and technical complexity.
5. **"Image-Based Crop Disease Detection Using Federated Learning"** (Nature, 2023)
- a. **Objective:** Explored privacy-preserving federated learning for collaborative crop disease detection.
 - b. **Relevance:** Demonstrated a secure and scalable method for training AI models on decentralized data.

2.3 PROPOSED SYSTEM

The proposed system is designed to revolutionize soil health monitoring and crop disease detection by leveraging advanced technologies such as machine learning, satellite imagery, and web-based analytics. It addresses the limitations of existing systems by offering accurate insights, user-friendly interfaces, and accessibility for farmers of varying scales. Below are the key components of the system:

1. High-Resolution Satellite Imagery

- **Source Platforms:** Utilize platforms like Google Earth Engine and Sentinel Hub to acquire high-resolution multispectral satellite imagery.
- **Data Analysis:** Analyze vegetation indices (e.g., NDVI, EVI) and spectral signatures to evaluate crop health, detect stress areas, and identify anomalies in agricultural fields.
- **Advantages:** Enables large-scale monitoring with minimal on-site intervention, covering extensive agricultural lands efficiently.

2. Soil Data Analysis

- **Soil Test Integration:** Leverage farmer-provided soil test data to assess soil quality, nutrient imbalances, and other factors affecting crop health.
- **Data Insights:** Analyze soil texture, organic matter, and nutrient levels (e.g., nitrogen, phosphorus, potassium) to recommend soil-specific actions.
- **Advantages:** Enhances understanding of soil properties, enabling informed decision-making for sustainable farming.

3. Machine Learning Models

- **Disease Detection:** Implement machine learning models like Convolutional Neural Networks (CNNs) to analyze disease-specific soil and satellite data, identifying potential outbreaks early.
- **Predictive Insights:** Use Random Forest classifiers and regression models to forecast yield trends and predict risks related to soil and environmental conditions.
- **Customizability:** Adapt algorithms for regional and crop-specific variations using localized training data.

4. Integration with Google Earth Engine

- **Satellite Data Processing:** Utilize Google Earth Engine for large-scale analysis of high-resolution imagery to track crop health, soil conditions, and potential disease patterns.
- **Real-Time Insights:** Process spatial data quickly, enabling near-instant feedback to users.
- **Scalability:** Supports small-scale farms and industrial-scale operations with consistent performance.

5. User-Friendly Web Application

- **Interactive Interface:** A responsive web-based application designed to display soil health, crop conditions, and analytics using interactive dashboards and maps.
- **Core Features:**
 - Soil test result visualization.
 - Disease detection alerts and recommendations for soil improvement.
 - Analysis of soil texture, erosion risk, and compaction.

- **Accessibility:** Simple navigation ensures that farmers with minimal technical expertise can utilize the platform effectively.

6. Soil Texture and Risk Analysis

- **Texture Classification:** Classify soil into types like sandy, silty, clayey, or loamy based on proportions of sand, silt, and clay.
- **Risk Assessment:** Identify risks such as erosion and compaction based on soil depth, texture, and organic matter levels.
- **Recommendations:** Provide actionable insights to mitigate identified risks, such as adding organic matter for sandy soils or adopting contour plowing for high erosion areas.

7. Data-Driven Recommendations

- **Optimization Tools:** Use soil and satellite data to recommend optimal farming practices, including nutrient management and irrigation scheduling.
- **Localized Support:** Tailor recommendations based on regional soil and environmental conditions for maximum impact.
- **Sustainability Focus:** Encourage efficient use of water, fertilizers, and pesticides to minimize environmental impact.

8. Scalable and Cost-Effective Design

- **Affordability:** Use open-source tools and an SQLite database for data management, reducing infrastructure costs.
- **Scalability:** Ensure compatibility across varying farm sizes, enabling both smallholder farmers and large-scale operations to benefit from the system.
- **Cloud-Based API Access:** Employ Google Earth Engine API for efficient satellite data processing without the need for complex infrastructure.

9. Educational Features

- **Knowledge Sharing:** Provide educational resources, including best practices for soil health management and disease prevention.
- **Community Support:** Introduce forums for farmers to discuss challenges, share insights, and learn collaboratively.

Anticipated Benefits

The proposed system addresses the challenges of traditional methods and delivers the following advantages:

1. **Early Detection and Prevention:** Enables timely interventions through real-time monitoring and machine learning insights, reducing crop losses.
2. **Comprehensive Analysis:** Combines soil and satellite data for a holistic understanding of agricultural conditions.
3. **User Accessibility:** Offers a simple, intuitive interface that ensures usability for farmers with varying levels of expertise.
4. **Sustainability:** Promotes eco-friendly farming by optimizing the use of natural resources and reducing reliance on chemical inputs.
5. **Economic Impact:** Boosts productivity while lowering operational costs, enhancing profitability for farmers.

By integrating these components, the project supports sustainable farming practices and contributes to global food security, aligning with sustainable development goals.

CHAPTER-3

ANALYSIS OF REVIEWED PAPERS

This chapter presents a detailed analysis of the reviewed papers, highlighting their methodologies, contributions, limitations, and how they relate to the proposed system. The reviewed papers collectively address critical aspects of crop health monitoring and disease detection, forming the foundation for the design and development of the proposed system.

3.1 REVIEW OBJECTIVES

The objective of reviewing these papers was to:

1. Understand the current state-of-the-art technologies in precision agriculture.
2. Identify gaps and limitations in existing approaches.
3. Synthesize insights for developing a robust, scalable, and user-friendly crop health monitoring system.

3.2 ANALYSIS OF PAPERS

1. Base Paper: A Convolutional Neural Network Approach to Robust Crop Health Monitoring (2024)

a. Key Contributions:

- i. Introduced CNNs for disease detection with 96% accuracy across 38 disease categories.
- ii. Integrated IoT sensor data with CNN outputs for enhanced insights.

b. Limitations:

- i. Focused primarily on disease classification without severity estimation.
- ii. Scalability challenges for diverse crops and regions.

c. **Relevance:**

- i. Provides a foundation for the disease detection module in the proposed system.

2. Machine Learning for Detection and Prediction of Crop Diseases (MDPI, 2022)

a. **Key Contributions:**

- i. Dual application of ML for detection and prediction of crop diseases.
- ii. Highlighted the importance of time-series data for predictive analytics.

b. **Limitations:**

- i. Dataset diversity and generalization across regions were limited.

c. **Relevance:**

- i. Informs the predictive analytics module for disease outbreak forecasting.

3. Early Detection of Crop Diseases Using CNN Classification (2023)

a. **Key Contributions:**

- i. Demonstrated the importance of early disease detection using CNNs.
- ii. Focused on minimizing yield losses through timely interventions.

b. **Limitations:**

- i. Overfitting issues in variable environmental conditions.

c. **Relevance:**

- i. Highlights the need for robust datasets and generalizable models.

4. Precision Agriculture with AI: Data Fusion and Advanced Analytics (Springer, 2024)

a. **Key Contributions:**

- i. Proposed a data fusion model combining IoT, satellite imagery, and historical data.
- ii. Addressed the challenges of integrating multi-modal datasets.

b. **Limitations:**

- i. Complexity in combining data streams for real-time decision-making.

c. **Relevance:**

- i. Informs the system's architecture and integration of diverse data sources.

5. Recent Advances in Crop Disease Detection Using UAV and Deep Learning Techniques (2023)

a. Key Contributions:

- i. Explored UAVs for localized and precise disease detection.
- ii. Discussed the application of multispectral sensors in agriculture.

b. Limitations:

- i. High operational costs and technical barriers to adoption.

c. Relevance:

- i. Complements the satellite-based data collection approach with UAV-based precision.

6. Image-Based Crop Disease Detection Using Federated Learning (Nature, 2023)

a. Key Contributions:

- i. Pioneered a privacy-preserving federated learning approach for disease detection.
- ii. Enabled collaborative model training without data centralization.

b. Limitations:

- i. High computational requirements for federated learning.

c. Relevance:

- i. Offers insights into secure data sharing for future iterations of the system.

3.3 GAPS IDENTIFIED

1. Scalability:

- a. Many reviewed systems faced challenges in adapting to diverse crops, climates, and geographical regions.

2. Integration of Multi-Modal Data:

- a. Limited studies effectively combined satellite imagery, IoT data, and historical records into a unified system.

3. Real-Time Analysis:

- a. Delays in data processing and transmission were common in existing solutions.

4. User Accessibility:

- a. Few systems prioritized ease of use for non-technical users, especially smallholder farmers.

5. Cost-Effectiveness:

- a. High costs of implementation limited the accessibility of many systems.

3.4 SYNTHESIS AND INSIGHTS

The reviewed papers collectively emphasize the importance of:

- **Data-Driven Insights:** Leveraging machine learning and data fusion for actionable recommendations.
- **Timeliness:** Ensuring real-time alerts and interventions to minimize yield losses.
- **Scalability and Accessibility:** Designing solutions that cater to diverse user bases, including small-scale farmers.
- **Sustainability:** Promoting eco-friendly farming practices through optimized resource utilization.

The analysis of reviewed papers highlights the strengths and limitations of existing approaches, providing valuable insights for the development of the proposed system. By addressing the identified gaps, the system aims to combine advanced technologies with practical usability, paving the way for a scalable, cost-effective, and impactful solution in precision agriculture.

CHAPTER-4

REQUIREMENT ANALYSIS

4.1 FUNCTIONAL REQUIREMENTS

1. Data Acquisition:

- The system must acquire high-resolution satellite imagery using APIs like Google Earth Engine for vegetation monitoring.
- Soil test data, including pH, nutrient levels, and soil composition, must be integrated for comprehensive analysis.

2. Data Processing:

- The system must preprocess satellite images to calculate vegetation indices such as NDVI and EVI for crop health assessment.
- Soil data must be normalized, cleaned, and combined with satellite insights to evaluate crop and soil conditions.

3. Disease Detection and Prediction:

- Machine learning models, including Convolutional Neural Networks (CNNs), should detect crop diseases from satellite imagery and soil data.
- Predictive models should forecast potential risks such as nutrient deficiencies, erosion, and yield trends using historical and current datasets.

4. User Interaction:

- The system must provide a user-friendly web application for farmers to access visualizations, soil test results, and actionable insights.
- Recommendations for improving soil health, preventing erosion, and mitigating compaction must be included.

5. Alerts and Notifications:

- Generate timely alerts for detected anomalies in soil or crop health.
- Notify users about recommended actions, such as adding organic matter or implementing erosion control measures.

4.2 NON-FUNCTIONAL REQUIREMENTS**1. Performance:**

- The system must process satellite and soil data with minimal latency, ensuring insights are delivered promptly.
- Machine learning inference for disease detection should operate efficiently for seamless user experience.

2. Scalability:

- The system must accommodate increased data loads, whether from additional satellite imagery or soil tests, without compromising performance.

3. Reliability:

- Ensure 99.9% uptime for the web application to maintain user trust.
- Data storage and processing must include fault-tolerant mechanisms to prevent data loss or corruption.

4. Usability:

- The web application must be compatible with both desktop and mobile devices, offering a simple and intuitive interface.

5. Security:

- Implement secure authentication and role-based access controls to safeguard user data.
-

- Ensure sensitive information, such as soil test results, is encrypted during storage and transmission.

6. Maintainability:

- Follow a modular design for code, ensuring easy updates and debugging.
- Provide thorough documentation for smooth integration and future enhancements.

4.3 DOMAIN AND UI REQUIREMENTS**1. Domain Requirements:**

- The system must function within precision agriculture, emphasizing crop health analysis and soil condition monitoring.
- Integrate soil, satellite, and historical data for comprehensive agricultural insights.

2. UI Requirements:

- Provide a dashboard displaying soil analysis, crop health status, and actionable insights.
- Include interactive maps for visualizing risk areas and resource optimization.
- Ensure a responsive, user-friendly interface compatible with desktops and mobile devices.

4.4 HARDWARE REQUIREMENTS

A personal computer with the following minimum specifications:

- Processor: Intel Core i3 or equivalent.
- RAM: Minimum 8 GB to handle data processing and machine learning tasks.
- Storage: At least 256 GB SSD or 500 GB HDD for storing datasets and software tools.

- GPU: Integrated graphics for basic processing (dedicated GPU optional for enhanced performance).

4.5 SOFTWARE REQUIREMENTS

The system relies on essential and specialized tools to ensure functionality and efficiency. The operating system should be either Windows 10/11 or Linux (Ubuntu 20.04 or later) to support necessary development and deployment tools.

Programming Languages and Frameworks:

- Python: For data preprocessing, machine learning model development, and backend processes.
- Flask: For building the web application backend.
- HTML, CSS, and JavaScript: To design and style the user interface.

Libraries and APIs:

- Machine Learning and Data Processing: PyTorch for model development, Scikit-learn for classical machine learning, and Pandas/NumPy for data manipulation.
- Image Processing: OpenCV for extracting features from crop and soil images.
- Satellite Data Access: Google Earth Engine API for acquiring and analyzing satellite imagery.
- Visualization: Matplotlib and Seaborn for creating static plots; Plotly for interactive visualizations.

Database and Storage:

- SQLite: For managing user data, soil test results, and recommendations.
- Google Cloud: Utilized specifically for Google Earth Engine API integration.

Development Tools:

- IDEs: PyCharm or Visual Studio Code for efficient development and debugging.
- Version Control: Git, hosted on platforms like GitHub for managing code repositories.
- Prototyping: Jupyter Notebook for experimenting with machine learning models.

Testing Tools:

- Selenium: For automated testing of the web application.
- Manual Testing: For verifying functionality across devices.

Additional Considerations:

The system must be scalable to accommodate future expansions, such as higher-resolution satellite data or additional predictive features. Cost efficiency is achieved by leveraging open-source tools and free-tier services during development. The web application is designed for cross-platform usability, ensuring accessibility across desktops, tablets, and smartphones.

CHAPTER 5

SYSTEM DESIGN

5.1 DESIGN GOALS

1. Scalability: Design the system to accommodate both smallholder and large-scale farms, ensuring adaptability to various data volumes and farm sizes.
2. Accuracy: Utilize advanced machine learning models like Random Forest classifiers and CNNs to achieve precise disease detection and crop health monitoring.
3. User-Friendliness: Create a responsive and interactive web application that is intuitive and accessible, even for users with minimal technical expertise.
4. Cost-Effectiveness: Leverage open-source tools and locally stored SQLite databases to minimize costs while maintaining efficiency and reliability.
5. Sustainability: Enhance sustainable farming practices by providing recommendations that optimize the use of water, fertilizers, and pesticides, reducing environmental impact.
6. Integration: Efficiently merge satellite imagery, soil test data, and historical records into a cohesive platform for comprehensive agricultural analysis.

5.2 SYSTEM ARCHITECTURE

The system design for crop health monitoring and disease detection is structured to enable efficient integration of data acquisition, processing, and user interaction. By utilizing advanced technologies such as satellite imagery, machine learning, and soil analysis, the system provides a robust and scalable solution tailored to modern agricultural needs. The architecture is divided into functional layers, each with distinct responsibilities:

The architecture comprises the following key layers

Data Acquisition Layer:

This layer collects inputs from diverse sources, including high-resolution satellite imagery accessed through the Google Earth Engine API, soil test data, and historical crop performance records. The imagery provides crucial insights, such as vegetation indices (e.g., NDVI), while soil tests offer data on nutrient levels and texture. Historical records aid in understanding long-term patterns and trends.

Data Processing Layer:

The processing layer transforms raw data into meaningful insights. Satellite images are preprocessed with techniques like cloud masking and vegetation index calculations to enhance clarity and relevance. Soil test data is cleaned, normalized, and cross-referenced with satellite data for comprehensive analysis. This layer also integrates diverse datasets, ensuring consistency and accuracy.

Machine Learning Layer:

This layer powers the system's analytical capabilities. Machine learning models, including Random Forest classifiers, are trained to detect diseases and identify nutrient deficiencies from soil and crop data. Convolutional Neural Networks (CNNs) analyze image data for disease patterns, while predictive models estimate crop yield and potential disease outbreaks based on real-time and historical inputs.

Storage Layer:

The system's data management needs are handled by this layer. SQLite databases store soil data, user profiles, and processed results. The integration of Google Cloud Platform ensures access to high-resolution imagery via the Google Earth Engine API, enabling scalable data processing without external cloud storage dependencies.

Application Layer:

This layer bridges the backend and the user interface. Built using Python Flask, the backend facilitates efficient communication with the database and processes user requests. The web-based frontend is developed with responsive HTML and CSS, offering interactive features such as crop health visualizations, alerts for detected diseases, and actionable recommendations.

Finally, the User Interface Layer ensures accessibility for end-users. Farmers can interact with the system through a web application, which displays crop health analytics, interactive maps, and actionable insights in an intuitive format. The application is by integrating these layers, the system provides a comprehensive solution for modern agricultural challenges, empowering farmers with actionable insights and fostering sustainable-practices.

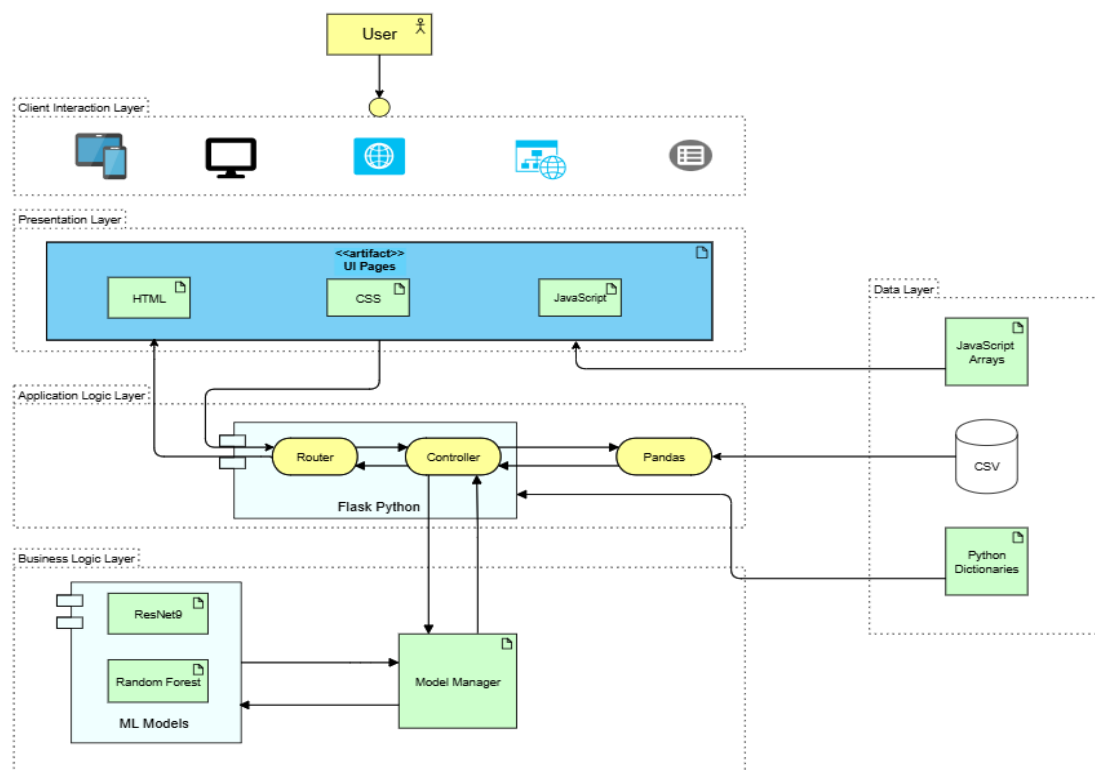


Fig 5.2.1 – System Design

5.3 ALGORITHMS AND WORKFLOW

The system employs tailored algorithms for data preprocessing, machine learning model training, and real-time analysis.

Data Preprocessing Algorithm:

This algorithm processes raw satellite imagery and soil test data. Techniques like cloud masking are applied to satellite images to remove noise and improve clarity. Vegetation indices such as NDVI are calculated to assess crop health, while soil test data is cleaned and normalized to ensure consistency. The processed data is then integrated into a unified dataset for comprehensive analysis.

Machine Learning Algorithm:

The machine learning models, including Random Forest classifiers, are utilized for disease detection. During the training phase, preprocessed datasets are used to train these models to recognize patterns associated with specific crop diseases or nutrient deficiencies. For real-time analysis, the trained model processes new soil and crop data, delivering accurate predictions and classifications to support informed decision-making.

Predictive Analytics Algorithm:

The system employs predictive analytics to forecast disease outbreaks and crop yields. Historical soil and crop data, combined with real-time satellite imagery, are used to train time-series models and machine learning algorithms. These models provide actionable insights, enabling farmers to anticipate potential challenges and optimize resource utilization effectively.

This integrated algorithmic approach ensures accurate, scalable, and timely analysis, empowering farmers with actionable insights to enhance agricultural productivity and sustainability.

The system's workflow is visually represented in the following stages:

1. **Data Input:** High-resolution satellite imagery is accessed via Google Earth Engine APIs, and soil test data is retrieved from the database.
2. **Preprocessing:** Satellite images are processed with techniques like cloud masking, while soil data is cleaned and normalized to ensure consistency and accuracy.
3. **Feature Extraction:** Key metrics such as vegetation indices (e.g., NDVI) are computed from satellite imagery, and soil health parameters are analyzed.
4. **Model Application:** Machine learning models, such as Random Forest classifiers, analyze the processed data to detect crop diseases, nutrient deficiencies, and predict yield trends.
5. **Output Delivery:** Results are presented on the user-friendly web interface, featuring interactive maps, visualizations, alerts, and actionable recommendations to assist farmers in decision-making.

5.4 DESIGN PRINCIPLES

The system design is guided by the following principles to ensure efficiency, usability, and adaptability:

1. **Scalability:** The architecture supports the integration of additional features, larger datasets, and diverse agricultural scenarios without performance degradation.
 2. **Modularity:** Each functional layer is designed as an independent module, enabling seamless updates, debugging, and maintenance while ensuring reusability across different system components.
 3. **User-Centricity:** The system emphasizes an intuitive interface tailored to the needs of farmers, ensuring accessibility and ease of use, regardless of technical expertise.
-

4. **Accuracy and Precision:** By leveraging advanced machine learning models and high-resolution satellite imagery, the system delivers reliable insights into crop health and disease detection.

5. **Real-Time Analysis:** The integration of satellite imagery and soil data allows the system to process and deliver actionable insights in near real-time, enabling timely interventions for optimal farming outcomes.

6. **Sustainability:** The design promotes environmentally friendly practices by optimizing resource usage and minimizing waste, aligning with global sustainability goals.

5.5 DATABASE DESIGN

The database design employs SQLite as the Database Management System (DBMS) to manage structured and relational data efficiently. The schema is designed to support user management, farmland records, soil test data, crop details, and integration with machine learning outputs.

Key Tables and Their Structure

1. Users Table

- Purpose: Stores details of users, including farmers and administrators.
- Columns:
 - `id` (Primary Key): Unique identifier for each user.
 - `username` (VARCHAR): Name of the user.
 - `email` (VARCHAR): Contact email address.
 - `password` (VARCHAR): Hashed password for authentication.
 - `role\_id` (Foreign Key): Links to the `Roles Table`.

2. Roles Table

- Purpose: Defines different user roles for access control.

- Columns:

- `id` (Primary Key): Unique identifier for each role.

- `description` (VARCHAR): Role type (e.g., Admin, Farmer, Manager).

3. Crops Table

- Purpose: Stores information about various crops.

- Columns:

- `id` (Primary Key): Unique identifier for each crop.

- `description` (VARCHAR): Name or type of the crop.

4. Farmlands Table

- Purpose: Maintains details of farmland records.

- Columns:

- `id` (Primary Key): Unique identifier for each farmland.

- `name` (VARCHAR): Name of the farmland.

- `croptype\_id` (Foreign Key): Links to the `Crops Table`.

- `sow\_date` (DATE): Date when the crop was sown.

- `harvest\_date` (DATE): Expected date of harvest.

- `production\_expected` (FLOAT): Predicted yield (kg/ha).

- `coordinates` (TEXT): Geographical coordinates of the farmland.

5. Soil Test Table

- Purpose: Stores results of soil tests performed on farmlands.

- Columns:

- `id` (Primary Key): Unique identifier for each test.

- `test\_name` (VARCHAR): Name or description of the soil test.
- `soil\_date` (DATE): Date when the test was conducted.
- `soil\_depth` (FLOAT): Depth of the soil sample (cm).
- `organic\_level` (FLOAT): Organic matter content (%).
- `phosphorus` (FLOAT): Phosphorus level (mg/dm³).
- `potassium` (FLOAT): Potassium level (mg/dm³).
- `calcium` (FLOAT): Calcium level (mg/dm³).
- `sand` (FLOAT): Sand content (%).
- `silt` (FLOAT): Silt content (%).
- `clay` (FLOAT): Clay content (%).
- `boron` (FLOAT): Boron level (mg/dm³).
- `copper` (FLOAT): Copper level (mg/dm³).
- `zinc` (FLOAT): Zinc level (mg/dm³).
- `manganese` (FLOAT): Manganese level (mg/dm³).

6. Disease Detection Table

- Purpose: Logs disease predictions for crops.
 - Columns:
 - `id` (Primary Key): Unique identifier for each record.
 - `farmland\_id` (Foreign Key): Links to the `Farmlands Table`.
 - `disease\_name` (VARCHAR): Name of the detected disease.
 - `confidence\_score` (FLOAT): Model's confidence in the prediction (%).
 - `date\_detected` (DATE): Date of disease detection.
-

7. Recommendations Table

- Purpose: Stores actionable insights for farmers.
- Columns:
 - `id` (Primary Key): Unique identifier for each recommendation.
 - `farmland\_id` (Foreign Key): Links to the `Farmlands Table`.
 - `recommendation\_type` (VARCHAR): Type of recommendation (e.g., Fertilizer, Pesticide).
 - `details` (TEXT): Description of the recommendation.
 - `date\_issued` (DATE): Date when the recommendation was generated.

Data Integration

Kaggle Datasets: Utilized for training models for disease detection and nutrient deficiency analysis. Relevant data includes:

- Crop disease images for CNN training.
- Soil and crop-specific data for nutrient recommendations.
- Historical trends for predictive analytics.

This relational database schema ensures efficient data management and supports the system's core functionalities, such as user authentication, farmland tracking, soil analysis, and disease detection.

5.6 USER INTERACTION FLOW

1. **Login/Authentication:**

- a. Users log in using secure credentials.
- b. Multi-level access for farmers, agronomists, and administrators.

2. **Dashboard:**

- a. **Overview Panel:** Displays crop health summaries and key metrics.
- b. **Alerts Section:** Shows notifications for detected diseases or abnormalities.

3. **Data Visualization:**

- a. Interactive maps for satellite imagery and vegetation indices.
- b. Charts for IoT sensor readings and historical trends.

4. **Disease Detection Module:**

- a. Allows farmers to upload crop images or view AI-detected results.
- b. Provides a list of probable diseases with recommendations for treatment.

5. **Recommendations:**

- a. Personalized suggestions for optimal use of water, fertilizers, and pesticides.
- b. Predictive analytics for upcoming challenges (e.g., potential disease outbreaks).

6. **Feedback and Community Forum:**

- a. Farmers can share experiences, ask questions, and access knowledge resources.

7. **Logout:**

- a. Ends the session and ensures data security.

The proposed system design ensures robust data integration and analysis, enabling real-time and predictive crop health monitoring. By combining advanced algorithms with a user-friendly interface, the design addresses key challenges in modern agriculture, paving the way for efficient and sustainable farming practices.

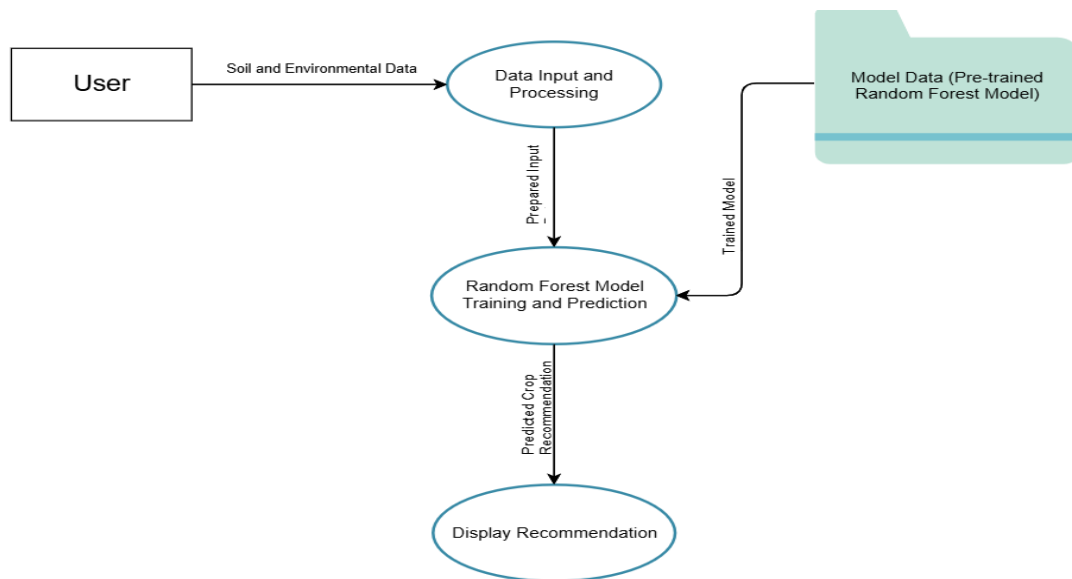


Fig 5.6.1 - Crop recommendation model design

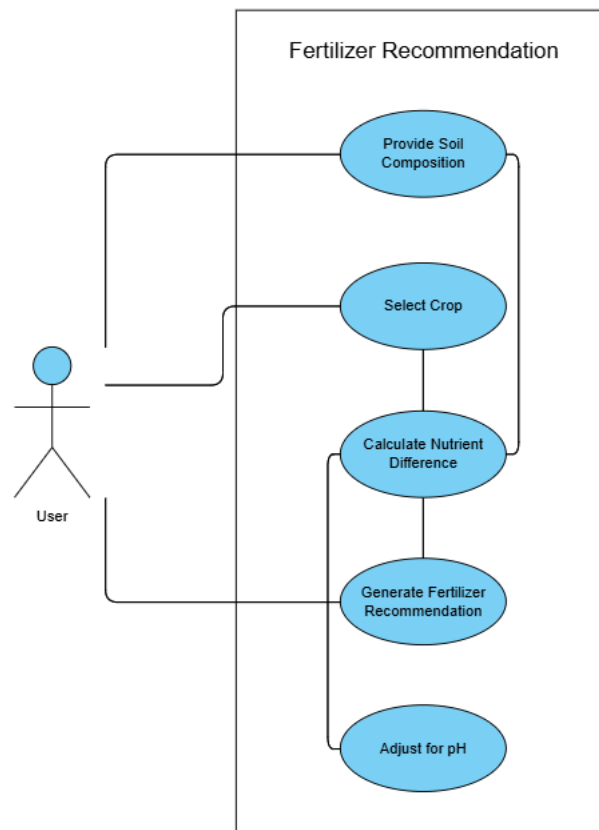


Fig. 5.6.2 - Fertilizer recommendation model design

5.7 FLOW CHARTS

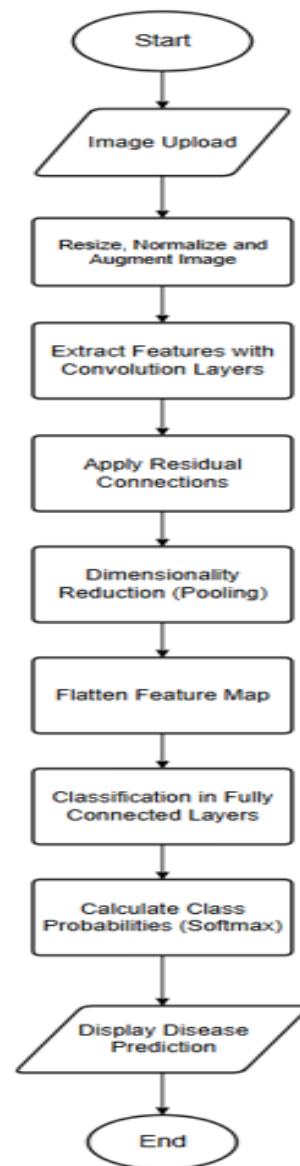


Fig 5.7.1 - Disease detection design

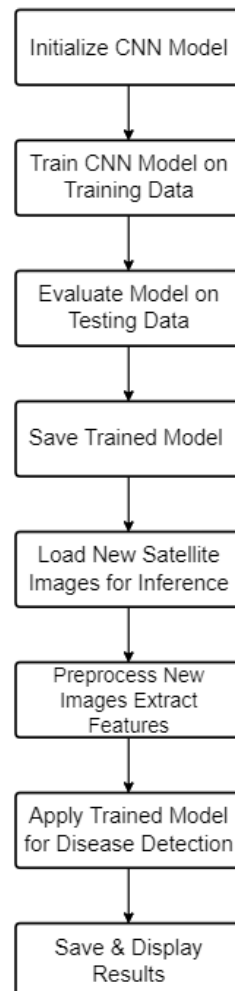


Fig 5.7.2 – CNN Model Training

CHAPTER-6

IMPLEMENTATION

6.1 ARCHITECTURE AND INFRASTRUCTURE

The implementation of the system is designed to support crop health monitoring and disease detection with an efficient architecture and scalable infrastructure. The architecture ensures seamless integration of various components for data acquisition, analysis, and user interaction.

1. System Components

a. Frontend

- Built using HTML, CSS, and JavaScript to provide a simple, responsive, and user-friendly web interface.
- Visualizations and interactive maps are implemented to allow farmers to explore crop health, disease insights, and soil test results.

b. Backend

- Developed using Flask, a Python web framework, to manage business logic and handle API requests.
- Ensures secure communication between the frontend, database, and external APIs.

c. Machine Learning Models

- Implemented in Python using frameworks like PyTorch and Scikit-learn .
- Models include:
 - Convolutional Neural Networks (CNNs): For detecting crop diseases using preprocessed image datasets.
 - Random Forest Classifiers: For predictive analytics, including crop yield estimation and disease outbreak forecasts.

d. Database

- SQLite: A lightweight relational database used for managing user profiles, soil data, crop records, and recommendations.
- Data is structured to allow efficient queries and updates.

e. Cloud Services

- Google Cloud Platform (GCP): Utilized to access the Google Earth Engine API key for satellite imagery analysis.
- Google Earth Engine (GEE): Provides access to high-resolution multispectral satellite images for vegetation indices (e.g., NDVI) and stress detection.

2. Infrastructure

a. Data Sources

- Kaggle Datasets: Pre-labeled datasets for soil properties, crop health, and disease patterns.
- Google Earth Engine: Supplies real-time satellite imagery for monitoring crop health and environmental factors.

b. Computational Resources

Development and model training are performed on personal computers equipped with:

- Processor: Intel Core i5 or higher.
- RAM: 8 GB or more for efficient computation.
- Storage: Minimum 256 GB SSD for fast access to datasets and model files
- Advanced model training can utilize cloud GPUs if required.

c. Network Requirements

- Reliable internet connectivity to ensure uninterrupted access to external APIs (e.g., GEE) and for seamless data transfer between system components.

3. Integration Approach

a. Data Acquisition

- Satellite imagery and soil data are fetched via Google Earth Engine and stored in the SQLite database for local access and preprocessing.

b. Data Processing and Analysis

- Machine learning models process satellite data, soil parameters, and crop records to generate actionable insights.
- The backend integrates these insights and makes them available to the web application.

c. User Interaction

- Farmers access the web application to visualize data through maps, charts, and reports.
- Alerts and recommendations are generated in real-time to guide farming decisions.

This architecture ensures efficiency, scalability, and cost-effectiveness, providing an accessible platform for farmers to make informed agricultural decisions. It aligns with the goals of precision agriculture by integrating advanced technology with practical farming solutions.

6.2 CHALLENGES DURING DEVELOPMENT

1. Data Integration

- a. Integrating diverse data sources, including soil test results, satellite imagery from Google Earth Engine, and Kaggle datasets, posed challenges in ensuring seamless compatibility and data consistency.
- b. Maintaining synchronization between static historical datasets and real-time satellite data for accurate analysis.

2. Machine Learning Model Generalization

- a. Developing models capable of handling diverse crop types, soil conditions, and climatic variations while ensuring robustness across regions.
- b. Avoiding overfitting by using appropriate data augmentation and ensuring sufficient training data diversity from Kaggle datasets.

3. Infrastructure Constraints

- a. Processing high-resolution satellite imagery and implementing machine learning algorithms required significant computational resources, which was challenging on local hardware setups.
- b. Ensuring all system modules, including data acquisition, analysis, and visualization, function cohesively during real-time operations.

4. User Accessibility

- a. Designing a web interface that is user-friendly and intuitive for farmers with limited technical expertise while ensuring quick access to actionable insights.
- b. Addressing the lack of multi-language support to make the application accessible to farmers in diverse regions.

5. Scalability and Cost Optimization

- a. Balancing system scalability to accommodate smallholder farms and larger agricultural operations without compromising performance.
- b. Minimizing development and operational costs by leveraging open-source tools, Google Earth Engine APIs, and lightweight databases like SQLite, ensuring affordability for small-scale farmers.

These challenges were addressed through iterative development, leveraging modular architecture, and focusing on practical, scalable, and cost-effective solutions to align with project goals.

6.3 MODULES IMPLEMENTED

1. Data Acquisition Module

- Purpose: To gather relevant data required for monitoring crop health and performing analyses.
- Google Earth Engine Integration: Accesses high-resolution satellite imagery to extract vegetation indices like NDVI and EVI.
- Soil Test Data: Collects detailed soil test information, including organic matter, macro and micro-nutrient levels, and soil depth.
- Historical Data: Utilizes historical crop and environmental data for trend analysis and predictions.

2. Data Preprocessing Module

- Purpose: To clean and process raw data for effective analysis.
- Satellite Image Processing: Applies cloud masking and computes vegetation indices to enhance image quality and extract meaningful insights.
- Soil Data Normalization: Cleans and standardizes soil data to ensure consistency in analysis.
- Data Integration: Combines satellite imagery, soil test results, and historical data into a unified dataset for further processing.

3. Disease and Nutrient Deficiency Detection Module

- Purpose: To identify diseases and nutrient imbalances in crops.
-

- Machine Learning Models: Implements Convolutional Neural Networks (CNNs) trained on Kaggle datasets for disease detection and classification.
- Nutrient Analysis: Detects deficiencies in macro and micro-nutrients based on soil test results, providing actionable insights for corrective measures.

4. Predictive Analytics Module

- Purpose: To predict future trends in crop health and potential disease outbreaks.
- Forecasting Models: Utilizes machine learning techniques like Random Forest and time-series analysis to predict yield trends and disease risks.
- Resource Optimization: Generates recommendations for optimal fertilizer application, irrigation scheduling, and pest control based on predictive insights.

5. User Interaction Module

- Purpose: To provide an accessible interface for farmers to visualize and interact with the system's outputs.
- Interactive Dashboard: Displays crop health analytics, vegetation indices, and soil test results through intuitive visualizations and maps.
- Alerts and Notifications: Provides real-time disease alerts and recommendations for treatment and resource management.
- Community Features: Includes a platform for farmers to share experiences, access educational resources, and seek support from peers.

6. Report Generation Module

- Purpose: To provide farmers with downloadable, detailed reports summarizing crop health and recommendations.
- Customized Reports: Generates tailored insights based on farm-specific data, including historical trends, nutrient recommendations, and disease analysis.
- Export Options: Allows reports to be downloaded in formats such as PDF for offline use and sharing.

These modules work cohesively to deliver a comprehensive solution for crop health monitoring and disease detection, emphasizing accuracy, scalability, and user accessibility.

6.4 ALGORITHM STEPS

Disease Detection Using CNN

1. Input:

- Crop images provided by farmers.
- Soil data from tests (macro and micro-nutrient levels, organic matter, etc.).
- Satellite imagery data acquired via Google Earth Engine.

2. Preprocessing:

- Image Processing:

- Enhance image quality by applying techniques such as resizing and normalization.
- Compute vegetation indices (e.g., NDVI, EVI) for satellite images to identify stressed areas.

- Data Cleaning:

- Standardize soil data and remove inconsistencies.
- Integrate soil and satellite data into a cohesive dataset.

3. Model Training:

Train the ResNet9 CNN Model on a labeled dataset of crop diseases sourced from Kaggle.

Steps:

1. Extract features (e.g., texture, color) using convolutional layers.
2. Reduce data dimensions while retaining essential features using pooling layers.
3. Use fully connected layers to classify diseases into predefined categories.

4. Inference:

- Input: Provide new crop images, soil data, and vegetation indices to the trained model.
- Prediction: The model identifies the disease class and its severity level based on visual and numerical patterns.

5. Output:

- Results: Disease classification, severity score, and suggested treatment or management actions.

Predictive Analytics Using Random Forest

1. Input:

- Historical soil and crop data.
- Real-time environmental parameters (e.g., soil moisture, temperature, pH).

2. Training:

- Train the Random Forest Model using historical trends and environmental variables.
- Incorporate time-series data to improve prediction accuracy.

3. Prediction:

Use the trained model to forecast:

1. Potential disease outbreaks.
2. Expected crop yield under current conditions.

4. Output:

- Forecasts:

- Probable onset of diseases.
- Yield estimates under various scenarios.

- Recommendations:

- Proactive measures to mitigate risks.
- Resource optimization strategies (e.g., irrigation schedules, fertilizer adjustments).

6.5 CODE

6.5.1 Crop data preparation:

The image displays three sequential screenshots of a Jupyter Notebook titled 'Crop_data_prep.ipynb' running in Google Colab. The notebook is used for preparing crop data by merging two datasets: 'crop' and 'fert'.

First Screenshot: Shows the initial data loading and inspection. The 'crop' dataset is loaded as a pandas Series with values like 'rice', 'rice', 'rice', 'rice', and 'watermelon'. The 'fert' dataset is loaded as a pandas DataFrame with columns 'Crop', 'Fertilizer', and 'Yield'.

```

crop['label']
0      rice
1      rice
2      rice
3      rice
4      rice
...
3895  watermelon
3896  watermelon
3897  watermelon
3898  watermelon
3899  watermelon
Name: label, length: 3100, dtype: object

extract_labels = []
for i in crop_names_from_fert:
    if i in crop_names:
        extract_labels.append(i)

# using extract_labels to get all the data related to those labels
new_crop = pd.DataFrame(columns = crop.columns)
new_fert = pd.DataFrame(columns = fert.columns)

for label in extract_labels:
    new_crop = new_crop.append(crop[crop['label'] == label])

for label in extract_labels:
    new_fert = new_fert.append(fert[fert['Crop'] == label].iloc[0])

new_crop

```

Second Screenshot: Shows the process of extracting unique crop names from the 'fert' dataset and creating a list of crop names.

```

del fert['Unnamed: 0']

crop_names_from_fert = fert['Crop'].unique()
crop_names_from_fert
array(['rice', 'jowar(sorghum)', 'barley(jav)', 'maize',
      'ragi(maachmili)', 'chickpea', 'frenchbeans(farasi)',
      'favabeans(papdi-val)', 'linabeans(pavta)', 'clusterbeans(gavar)',
      'soyabean', 'blackeyedbeans(chauli)', 'kidneybeans', 'pigeonpeas',
      'mudbean', 'mungbean', 'greenpeas', 'horsegram(bulhi)',
      'blackgram', 'rapeseed(mohri)', 'coriandersseeds', 'mustardseeds',
      'sesameseed', 'cuminseeds', 'lentil', 'brinjal', 'beetroot',
      'bittergourd', 'bottlegourd', 'capsicum', 'cabbage', 'carrot',
      'cauliflower', 'cucumber', 'corianderleaves', 'curryleaves',
      'drumstick-moringa', 'chilli', 'ladyfinger', 'mushrooms', 'onion',
      'potato', 'pumpkin', 'radish', 'olive', 'sweetpotato',
      'fenugreekleaf(methi)', 'spinach', 'ridgegourd',
      'gooseberry(amla)', 'jamun(syzygiumcumini)',
      'ziziphusauritiana(bo)', 'garciniaindica(kokam)', 'tamarind',
      'tapioca(furan)', 'garlic', 'lemon', 'tomato', 'ashgourd',
      'pineapple', 'pomegranate', 'banana', 'mango', 'grapes'])

```

Third Screenshot: Shows the final step of saving the prepared data into CSV files.

```

new_crop.to_csv('../Data-raw/MergeFileCrop.csv')
new_fert.to_csv('../Data-raw/fertilizerData.csv')

```

Fig 6.5.1.1 – Crop data preparation

6.5.2 CropSense Code

app.py

```

app.py 2 X
You, 19 hours ago | 2 authors (Gladiator07 and one other)
1 from flask import Flask, render_template, request, Markup, jsonify
2
3 import numpy as np
4 import pandas as pd
5 from utils.disease import disease_dic
6 from utils.fertilizer import fertilizer_dic
7 import requests
8 import config
9 import pickle
10 import io
11 import torch
12 from torchvision import transforms
13 from PIL import Image
14 from utils.model import ResNet9
15 from tensorflow.keras.models import load_model
16 from tensorflow.keras.preprocessing.image import load_img, img_to_array
17 from utils.nutrient import nutrient_dic
18
19 from flask import Flask, render_template, request, redirect, url_for, session
20 from flask_sqlalchemy import SQLAlchemy
21 import traceback
22
23 #LOADING THE TRAINED MODELS
24 #Loading plant disease classification model
25
26 disease_classes = ['Apple__Apple_scab',
27                   'Apple__Black_rot',
28                   'Apple__Cedar_apple_rust',
29                   'Apple__healthy',
30                   'Blueberry__healthy',
31                   'Cherry__(including_sour)__Powdery_mildew',
32                   'Cherry__(including_sour)__healthy',
33                   'Corn_(maize)__Cercospora_leaf_spot Gray_leaf_spot',
34                   'Corn_(maize)__Common_rust_',
35                   'Corn_(maize)__Northern_Leaf_Blight',
36                   'Corn_(maize)__healthy',
37                   'Grape__Black_rot',
38                   'Grape__Esca_(Black_Measles)',
39                   'Grape__Leaf_blight_(Isariopsis_Leaf_Spot)',
40                   'Grape__healthy',
41                   'Orange__Huanglongbing_(Citrus_greening)',
42                   'Peach__Bacterial_spot',
43                   'Peach__healthy',
44                   'Pepper,_bell__Bacterial_spot',
45                   'Pepper,_bell__healthy',
46                   'Potato__Early_blight',
47                   'Potato__Late_blight',
48                   'Potato__healthy',
49                   'Raspberry__healthy',
50                   'Soybean__healthy',
51                   'Squash__Powdery_mildew',
52                   'Strawberry__Leaf_scorch',
53                   'Strawberry__healthy',
54                   'Tomato__Bacterial_spot',
55                   'Tomato__Early_blight',
56                   'Tomato__Late_blight',
57                   'Tomato__Leaf_Mold',
58                   'Tomato__Septoria_leaf_spot',
59                   'Tomato__Spider_mites Two-spotted_spider_mite',
60                   'Tomato__Target_Spot',
61                   'Tomato__Tomato_Yellow_Leaf_Curl_Virus',
62                   'Tomato__Tomato_mosaic_virus',
63                   'Tomato__healthy']
64
65 disease_model_path = 'models/plant_disease_model.pth'
66 disease_model = ResNet9(3, len(disease_classes))
67 disease_model.load_state_dict(torch.load(
68     disease_model_path, map_location=torch.device('cpu'))))
69 disease_model.eval()
70
71
72 # Loading crop recommendation model
73 crop_recommendation_model_path = 'models/RandomForest.pkl'
74 crop_recommendation_model = pickle.load(
75     open(crop_recommendation_model_path, 'rb'))
76
77 # Custom functions for calculations
78 def weather_fetch(city_name):
79     """
80     Fetch and returns the temperature and humidity of a city
81     :params: city_name
82     :return: temperature, humidity
83     """
84     api_key = config.weather_api_key
85     base_url = "http://api.openweathermap.org/data/2.5/weather?"

```

```

87     complete_url = base_url + "appid=" + api_key + "&q=" + city_name
88     response = requests.get(complete_url)
89     x = response.json()
90     if x["cod"] != "404":
91         y = x["main"]
92
93         temperature = round((y["temp"] - 273.15), 2)
94         humidity = y["humidity"]
95         return temperature, humidity
96     else:
97         return None
98
99 def predict_image(img, model=disease_model):
100     """
101     Transforms image to tensor and predicts disease label
102     :param: image
103     :return: prediction (string)
104     """
105     transform = transforms.Compose([
106         transforms.Resize(256),
107         transforms.ToTensor(),
108     ])
109     image = Image.open(io.BytesIO(img))
110     img_t = transform(image)
111     img_u = torch.unsqueeze(img_t, 0)
112
113     # Get predictions from model
114     yb = model(img_u)
115     # Pick index with highest probability
116     _, preds = torch.max(yb, dim=1)
117     prediction = disease_classes[preds[0].item()]
118     # Retrieve the class label
119     return prediction
120
121 # Loading nutrient model
122 try:
123     model = load_model('utils/nutrient_model.h5')
124 except Exception as e:
125     print(f"Error loading model: {e}")
126
127 # Prediction function
128 def predict_nutrient(image_path):
129     img = load_img(image_path, target_size=(150, 150))
130     img_array = img_to_array(img) / 255.0
131     img_array = np.expand_dims(img_array, axis=0)
132
133     predictions = model.predict(img_array)
134     classes = ['Nitrogen', 'Phosphorus', 'Potassium']
135     predicted_class = classes[np.argmax(predictions)]
136     confidence = np.max(predictions) * 100
137     return predicted_class, confidence
138
139
140 # FLASK APP
141 app = Flask(__name__)
142
143 app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///users.db'
144 app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
145 app.secret_key = 'your_secret_key' # Set a secret key for session management
146
147 db = SQLAlchemy(app)
148
149 # User model
150 You, 21 hours ago | 1 author (You)
151 class User(db.Model):
152     id = db.Column(db.Integer, primary_key=True)
153     name = db.Column(db.String(100), nullable=False)
154     email = db.Column(db.String(100), nullable=False, unique=True)
155     password = db.Column(db.String(100), nullable=False)
156
157 @app.before_first_request
158 def create_tables():
159     db.create_all()
160
161
162 @app.route('/', methods=['GET', 'POST'])
163 def index():
164     if request.method == 'POST':
165         action_type = request.form.get('actionType') # Get the hidden field value
166         if action_type == 'signup': # Sign up form submission
167             try:
168                 name = request.form['name']
169                 email = request.form['email']
170                 password = request.form['password']
171

```

```

172         if not name or not email or not password:
173             return render_template('signup.html', message="All fields are required.")
174
175         # Check if the email already exists
176         existing_user = User.query.filter_by(email=email).first()
177         if existing_user:
178             return render_template('signup.html', message="Email already registered! Please log in.")
179
180         # Add new user
181         new_user = User(name=name, email=email, password=password)
182         db.session.add(new_user)
183         db.session.commit()
184         return render_template('signup.html', message="Sign up successful! Please log in.")
185     except Exception as e:
186         db.session.rollback() # Rollback transaction on error
187         print(f"Error occurred: {e}")
188         return render_template('signup.html', message=f"An error occurred: {str(e)}")
189     finally:
190         db.session.close() # Ensure session is closed
191 elif action_type == 'login': # Login form submission
192     email = request.form['email']
193     password = request.form['password']
194
195     user = User.query.filter_by(email=email).first()
196     if user and user.password == password:
197         session['user_id'] = user.id # Store user ID in session
198         session['user_name'] = user.name # Store user name in session
199         return redirect(url_for('home')) # Redirect to home page
200     else:
201         return render_template('signup.html', message="Invalid credentials. Please try again.")
202 return render_template('signup.html')
203
204
205
206
207
208 @app.route('/dashboard')
209 def dashboard():
210     if 'user_id' in session:
211         return f"Hello, {session['user_name']}! Welcome to your dashboard."
212     else:
213         return redirect(url_for('index'))
214
215 @app.route('/logout')
216 def logout():
217     session.pop('user_id', None) # Remove user_id if it exists
218     session.pop('user_name', None) # Remove user_name if it exists
219     return redirect(url_for('index')) # Redirect to the login page
220
221 # @app.route('/logout')
222 # def logout():
223 #     session.clear() # Clear session data
224 #     return redirect(url_for('login')) # Redirect to login page
225
226
227 # render home page
228 @app.route('/')
229 def home():
230     title = 'Crop Sense - Home'
231     return render_template('index.html', title=title)
232
233 # render crop recommendation form page
234 @app.route('/crop-recommend')
235 def crop_recommend():
236     title = 'Crop Sense - Crop Recommendation'
237     return render_template('crop.html', title=title)
238
239 # render fertilizer recommendation form page
240 @app.route('/fertilizer')
241 def fertilizer_recommendation(): # Gladiator07, 4 years ago • added endpoints
242     title = 'Crop Sense - Fertilizer Suggestion'
243
244     return render_template('fertilizer.html', title=title)
245
246 # Route for nutrient deficiency detection
247 @app.route('/nutrient')
248 def nutrient_deficiency():
249     title = 'Crop Sense - Nutrient Deficiency Detection'
250     return render_template('nutrient.html', title=title)
251
252
253 # render disease prediction input page
254
255 # RENDER PREDICTION PAGES
256 # render crop recommendation result page
257 @app.route('/crop-predict', methods=['POST'])

```

```

258 def crop_prediction():
259     title = 'Crop Sense - Crop Recommendation'
260
261     if request.method == 'POST':
262         N = int(request.form['nitrogen'])
263         P = int(request.form['phosphorous'])
264         K = int(request.form['pottasium'])
265         ph = float(request.form['ph'])
266         rainfall = float(request.form['rainfall'])
267
268         # state = request.form.get("stt")
269         city = request.form.get("city")
270
271         if weather_fetch(city) != None:
272             temperature, humidity = weather_fetch(city)
273             data = np.array([[N, P, K, temperature, humidity, ph, rainfall]])
274             my_prediction = crop_recommendation_model.predict(data)
275             final_prediction = my_prediction[0]
276
277             return render_template('crop-result.html', prediction=final_prediction, title=title)
278         else:
279             return render_template('try_again.html', title=title)
280
281 # render fertilizer recommendation result page
282 @app.route('/fertilizer-predict', methods=['POST'])
283 def fert_recommend():
284     title = 'Crop Sense - Fertilizer Suggestion'
285
286     crop_name = str(request.form['cropname'])
287     N = int(request.form['nitrogen'])
288     P = int(request.form['phosphorous'])
289     K = int(request.form['pottasium'])
290     # ph = float(request.form['ph'])
291
292     df = pd.read_csv('Data/fertilizer.csv')
293
294     nr = df[df['Crop'] == crop_name]['N'].iloc[0]
295     pr = df[df['Crop'] == crop_name]['P'].iloc[0]
296     kr = df[df['Crop'] == crop_name]['K'].iloc[0]
297
298     n = nr - N
299     p = pr - P
300     k = kr - K
301
302     temp = {abs(n): "N", abs(p): "P", abs(k): "K"}
303     max_value = temp[max(temp.keys())]
304     if max_value == "N":
305         if n < 0:
306             key = 'NHigh'
307         else:
308             key = "Nlow"
309     elif max_value == "P":
310         if p < 0:
311             key = 'PHigh'
312         else:
313             key = "Plow"
314     else:
315         if k < 0:
316             key = 'KHigh'
317         else:
318             key = "Klow"
319
320     response = Markup(str(fertilizer_dic[key]))
321
322     return render_template('fertilizer-result.html', recommendation=response, title=title)
323
324 # render disease prediction result page
325
326 @app.route('/disease-predict', methods=['GET', 'POST'])
327 def disease_prediction():
328     title = 'Crop Sense - Disease Detection'
329
330     if request.method == 'POST':
331         if 'file' not in request.files:
332             return redirect(request.url)
333         file = request.files.get('file')
334         if not file:
335             return render_template('disease.html', title=title)
336         try:
337             img = file.read()
338
339             prediction = predict_image(img)
340
341             prediction = Markup(str(disease_dic[prediction]))
342             return render_template('disease-result.html', prediction=prediction, title=title)
343         except:

```

```

344 |         pass
345 |         return render_template('disease.html', title=title)
346 |
347 |     ## Dictionary with detailed nutrient information
348 |     # nutrient_info = {
349 |     #     'Nitrogen': {
350 |     #         'description': 'Nitrogen is essential for the growth of plants and plays a key role in photosynthesis.',
351 |     #         'recommendation': 'Increase nitrogen-rich fertilizers such as urea, ammonium nitrate, etc.'
352 |     #     },
353 |     #     'Phosphorus': {
354 |     #         'description': 'Phosphorus is crucial for energy transfer and photosynthesis in plants.',
355 |     #         'recommendation': 'Use phosphorus-rich fertilizers such as superphosphate or bone meal.'
356 |     #     },
357 |     #     'Potassium': {
358 |     #         'description': 'Potassium helps plants build resistance to diseases and aids in root development.',
359 |     #         'recommendation': 'Use potassium-rich fertilizers like potassium chloride or potassium sulfate.'
360 |     #     }
361 |     # }
362 |
363 | @app.route('/nutrient', methods=['GET', 'POST'])
364 | def nutrient_prediction():
365 |     title = 'Crop Sense - Nutrient Deficiency Detection'
366 |
367 |     if request.method == 'GET':
368 |         return render_template('nutrient.html', title=title)
369 |
370 |     if request.method == 'POST':
371 |         try:
372 |             # Collect the uploaded image for nutrient analysis
373 |             if 'image' not in request.files:
374 |                 raise ValueError("No file part in the request")
375 |
376 |             file = request.files['image']
377 |             if file.filename == '':
378 |                 raise ValueError("No file selected for uploading")
379 |
380 |             # Save the uploaded image temporarily
381 |             image_path = 'temp_image.jpg'
382 |             file.save(image_path)
383 |
384 |             # Call the prediction function for nutrient deficiency
385 |             predicted_class, confidence = predict_nutrient(image_path)
386 |
387 |             # Get detailed information from nutrient_dic for the predicted nutrient
388 |             nutrient_details = nutrient_dic.get(predicted_class, "No details available.")
389 |
390 |             # Use Markup to format the result directly and pass it to the result page
391 |             result = Markup(f"""
392 |                 <p><b>{predicted_class}</b></p>
393 |                 <p><i>{nutrient_details}</i></p>
394 |                 <p><b>Confidence:</b> {confidence:.2f}%</p>
395 |             """)
396 |
397 |             # Render the result template with formatted result
398 |             return render_template(
399 |                 'nutrient-result.html',
400 |                 result=result, # Pass the formatted result directly
401 |                 title=title
402 |             )
403 |         except Exception as e:
404 |             print(f"Error: {e}")
405 |             return render_template(
406 |                 'nutrient.html',
407 |                 title=title,
408 |                 error="An error occurred while processing the image. Please try again."
409 |             )
410 |
411 |
412 | if __name__ == '__main__':
413 |     app.run(debug=True)
414 |
415 |

```

Fig 6.5.2.1 app.py code

train_model.py

```

train_model.py 4 •
You, 5 days ago | 1 author (You)
1 import os
2 import tensorflow as tf
3 from tensorflow.keras.preprocessing.image import ImageDataGenerator
4 from tensorflow.keras.models import Sequential
5 from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
6 from tensorflow.keras.optimizers import Adam
7
8 # Set the path to your dataset (update this path accordingly)
9 train_dir = r'C:\Users\mary6\OneDrive\Documents\Major Project\SmartFarm\Crop Sense\dataset'
10
11 # Image Preprocessing: Normalizing and augmenting images
12 train_datagen = ImageDataGenerator(
13     rescale=1./255, # Normalize pixel values to [0, 1]
14     horizontal_flip=True, # Randomly flip images
15     rotation_range=20 # Randomly rotate images
16 )
17
18 # Load training data from directory
19 train_generator = train_datagen.flow_from_directory(
20     train_dir,
21     target_size=(150, 150), # Resize images to 150x150
22     batch_size=32,
23     class_mode='categorical' # We have 3 classes: N, P, K
24 )
25
26 # Build CNN Model
27 model = Sequential([
28     Conv2D(32, (3, 3), activation='relu', input_shape=(150, 150, 3)),
29     MaxPooling2D((2, 2)),
30     Conv2D(64, (3, 3), activation='relu'),
31     MaxPooling2D((2, 2)),
32     Conv2D(128, (3, 3), activation='relu'),
33     MaxPooling2D((2, 2)),
34     Flatten(),
35     Dense(128, activation='relu'),
36     Dense(3, activation='softmax') # Output layer with 3 neurons (one for each class)
37 ])
38
39 # Compile the model
40 model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
41 # Train the model
42 model.fit(train_generator, epochs=10) # Train for 10 epochs (can adjust this based on your data)
43
44 # Save the trained model
45 model.save('nutrient_model.h5')

```

Fig 6.5.2.2 train_model.py Code

model.py

```

model.py x
You, 5 days ago | 2 authors (Gladiator07 and one other)
1 import torch
2 import torch.nn as nn
3 import torch.nn.functional as F
4
5
6 def ConvBlock(in_channels, out_channels, pool=False):
7     layers = [nn.Conv2d(in_channels, out_channels, kernel_size=3, padding=1),
8               nn.BatchNorm2d(out_channels),
9               nn.ReLU(inplace=True)]
10    if pool:
11        layers.append(nn.MaxPool2d(4))
12    return nn.Sequential(*layers)
13
14
15 # Model Architecture
16 You, 5 days ago | 2 authors (Gladiator07 and one other)
17 class ResNet9(nn.Module):
18     def __init__(self, in_channels, num_diseases):
19         super().__init__()
20
21         self.conv1 = ConvBlock(in_channels, 64)
22         self.conv2 = ConvBlock(64, 128, pool=True) # out_dim : 128 x 64 x 64
23         self.res1 = nn.Sequential(ConvBlock(128, 128), ConvBlock(128, 128))
24
25         self.conv3 = ConvBlock(128, 256, pool=True) # out_dim : 256 x 16 x 16
26         self.conv4 = ConvBlock(256, 512, pool=True) # out_dim : 512 x 4 x 44
27         self.res2 = nn.Sequential(ConvBlock(512, 512), ConvBlock(512, 512))
28
29         self.classifier = nn.Sequential(nn.MaxPool2d(4),
30                                         nn.Flatten(),
31                                         nn.Linear(512, num_diseases))
32
33     def forward(self, xb): # xb is the loaded batch
34         out = self.conv1(xb)
35         out = self.conv2(out)
36         out = self.res1(out) + out
37         out = self.conv3(out)
38         out = self.res2(out) + out
39         out = self.classifier(out)
40         return out
41

```

Fig 6.5.2.3 model.py Code

disease.py

```

disease.py x
Gladiator07, 4 years ago | 1 author (Gladiator07)
1 disease_dic = {
2     'Apple__Apple_scab': """ <b>Crop</b>: Apple <br/>Disease: Apple Scab<br/>
3     <br/>Cause of disease:
4
5     <br/><br/> 1. Apple scab overwinters primarily in fallen leaves and in the soil. Disease development is favored by wet, cool weather that generally occurs in
6
7     <br/> 2. Fungal spores are carried by wind, rain or splashing water from the ground to flowers, leaves or fruit. During damp or rainy periods, newly opening .
8     <br/><br/> How to prevent/cure the disease <br/>
9     <br/>1. Choose resistant varieties when possible.
10
11     <br/>2. Rake under trees and destroy infected leaves to reduce the number of fungal spores available to start the disease cycle over again next spring
12
13     <br/>3. Water in the evening or early morning hours (avoid overhead irrigation) to give the leaves time to dry out before infection can occur.
14     <br/>4. Spread a 3- to 6-inch layer of compost under trees, keeping it away from the trunk, to cover soil and prevent splash dispersal of the fungal spores."
15
16     'Apple__Black_rot': """ <b>Crop</b>: Apple <br/>Disease: Black Rot<br/>
17     <br/>Cause of disease:
18
19     <br/><br/>Black rot is caused by the fungus Diplodia seriata (syn Botryosphaeria obtusa).The fungus can infect dead tissue as well as living trunks, branches
20     <br/><br/> How to prevent/cure the disease <br/>
21     <br/>1. Prune out dead or diseased branches.
22
23     <br/>2. Prune out dead or diseased branches.
24
25     <br/>3. Remove infected plant material from the area.
26     <br/>4. Remove infected plant material from the area.
27     <br/>5. Be sure to remove the stumps of any apple trees you cut down. Dead stumps can be a source of spores.""",
28
29     'Apple__Cedar_apple_rust': """ <b>Crop</b>: Apple <br/>Disease: Cedar Apple Rust<br/>
30     <br/>Cause of disease:
31
32     <br/><br/>Cedar apple rust (Gymnosporangium juniperi-virginianae) is a fungal disease that depends on two species to spread and develop. It spends a portion .
33
34     <br/><br/> How to prevent/cure the disease <br/>
35     <br/>1. Since the juniper galls are the source of the spores that infect the apple trees, cutting them is a sound strategy if there aren't too many of them.
36
37     <br/>2. While the spores can travel for miles, most of the ones that could infect your tree are within a few hundred feet.
38
39     <br/>3. The best way to do this is to prune the branches about 4-6 inches below the galls.""",
40
41 }

```

Fig 6.5.2.4 disease.py code

fertilizer.py

```

fertilizer.py x
Gladiator07, 4 years ago | 1 author (Gladiator07)
1 fertilizer_dic = {
2     'NHigh': """The N value of soil is high and might give rise to weeds.
3     <br/> Please consider the following suggestions:
4
5     <br/><br/> 1. <i>Manure</i> ☐ adding manure is one of the simplest ways to amend your soil with nitrogen. Be careful as there are various types of manures .
6
7     <br/> 2. <i>Coffee grinds</i> ☐ use your morning addiction to feed your gardening habit! Coffee grinds are considered a green compost material which is rich
8
9     <br/>3. <i>Plant nitrogen fixing plants</i> ☐ planting vegetables that are in Fabaceae family like peas, beans and soybeans have the ability to increase nitr
10
11     <br/>4. Plant ☐green manure☐ crops like cabbage, corn and broccoli
12
13     <br/>5. <i>Use mulch (wet grass) while growing crops</i> - Mulch can also include sawdust and scrap soft woods""",
14
15     'Nlow': """The N value of your soil is low.
16     <br/> Please consider the following suggestions:
17     <br/><br/> 1. <i>Add sawdust or fine woodchips to your soil</i> ☐ the carbon in the sawdust/woodchips love nitrogen and will help absorb and soak up and exce
18
19     <br/>2. <i>Plant heavy nitrogen feeding plants</i> ☐ tomatoes, corn, broccoli, cabbage and spinach are examples of plants that thrive off nitrogen and will s
20
21     <br/>3. <i>Water</i> ☐ soaking your soil with water will help leach the nitrogen deeper into your soil, effectively leaving less for your plants to use.
22
23     <br/>4. <i>Sugar</i> ☐ In limited studies, it was shown that adding sugar to your soil can help potentially reduce the amount of nitrogen is your soil. Sugar
24
25     <br/>5. Add composted manure to the soil.
26
27     <br/>6. Plant Nitrogen fixing plants like peas or beans.
28
29     <br/>7. <i>Use NPK fertilizers with high N value.
30
31     <br/>8. <i>Do nothing</i> ☐ It may seem counter-intuitive, but if you already have plants that are producing lots of foliage, it may be best to let them cont
32
33     'PHigh': """The P value of your soil is high.
34     <br/> Please consider the following suggestions:
35
36     <br/><br/>1. <i>Avoid adding manure</i> ☐ manure contains many key nutrients for your soil but typically including high levels of phosphorous. Limiting the a
37
38     <br/>2. <i>Use only phosphorus-free fertilizer</i> ☐ if you can limit the amount of phosphorous added to your soil, you can let the plants use the existing p
39
40     <br/>3. <i>Water your soil</i> ☐ soaking your soil liberally will aid in driving phosphorous out of the soil. This is recommended as a last ditch effort.
41
42     <br/>4. Plant nitrogen fixing vegetables to increase nitrogen without increasing phosphorous (like beans and peas).

```

Fig 6.5.2.5 fertilizer.py code

nutrient.py

```

1  You, 5 days ago | 1 author (You)
2  nutrient_dic = {
3      'Nitrogen': """Nitrogen deficiency is commonly characterized by yellowing of the older leaves and stunted growth.
4      <br/> Please consider the following suggestions:
5
6      <br/><br/> 1. <i>Add organic compost or manure</i> ☐ Organic materials like compost and well-rotted manure are rich in nitrogen and can help improve soil fertili:
7
8      <br/> 2. <i>Plant nitrogen-fixing crops</i> ☐ Crops such as peas, beans, and clover can help add nitrogen back into the soil naturally.
9
10     <br/> 3. <i>Use urea or ammonium nitrate fertilizers</i> ☐ Chemical fertilizers containing nitrogen can quickly address nitrogen deficiencies in soil.
11
12     <br/> 4. <i>Mulch your soil</i> ☐ Organic mulch such as grass clippings or leaves can help conserve moisture and improve soil nitrogen content over time.""",
13
14      'Phosphorus': """Phosphorus deficiency typically leads to stunted growth, dark green leaves, and poor root development.
15      <br/> Please consider the following suggestions:
16
17      <br/><br/> 1. <i>Apply rock phosphate</i> ☐ This slow-release form of phosphorus is effective in correcting phosphorus deficiencies over time.
18
19      <br/> 2. <i>Use bone meal</i> ☐ Bone meal is a rich source of phosphorus that can be added to the soil to boost nutrient levels.
20
21      <br/> 3. <i>Compost with high phosphorus content</i> ☐ Some organic composts contain naturally occurring phosphorus that can enrich the soil.
22
23      <br/> 4. <i>Ensure proper pH levels</i> ☐ Phosphorus is most available to plants in soils with a pH between 6 and 7. Testing and adjusting soil pH can improve ph
24
25      'Potassium': """Potassium deficiency is usually observed as yellowing or browning of the leaf edges and weak stems.
26      <br/> Please consider the following suggestions:
27
28      <br/><br/> 1. <i>Use potassium sulfate or potassium chloride fertilizers</i> ☐ These are effective sources of potassium and can address deficiencies quickly.
29
30      <br/> 2. <i>Apply composted manure</i> ☐ Manure is rich in potassium and can help replenish the soil over time.
31
32      <br/> 3. <i>Use wood ash</i> ☐ Wood ash contains potassium and can help raise the potassium levels in the soil.
33
34      <br/> 4. <i>Increase organic matter</i> ☐ Organic matter can improve nutrient availability, including potassium, in the soil.""",
35
36      # Add other nutrient deficiencies as needed
37  }

```

Fig 6.5.2.6 nutrient.py code

Layout.html

```
layout.html X
You, 22 hours ago | 3 authors (Gladiator07 and others)
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <title>{{ title }}</title>
6   <link rel="shortcut icon" href="{{ url_for('static', filename='images/favicon.ico') }}" />
7
8   <!-- for-mobile-apps -->
9   <meta name="viewport" content="width=device-width, initial-scale=1">
10  <meta charset="utf-8">
11  <meta name="keywords" content="Agro Harvest Responsive web template, Bootstrap Web Templates, Flat Web Templates, Android Compatible web template,
12  Smartphone Compatible web template, free webdesigns for Nokia, Samsung, LG, SonyEricsson, Motorola web design" />
13
14  <style>
15    html {
16      font-size: 1rem;
17    }
18
19    @media (min-width: 576px) {
20      html {
21        font-size: 1.25rem;
22      }
23    }
24
25    @media (min-width: 768px) {
26      html {
27        font-size: 1.5rem;
28      }
29    }
30
31    @media (min-width: 992px) {
32      html {
33        font-size: 1.75rem;
34      }
35    }
36
37    @media (min-width: 1200px) {
38      html {
39        font-size: 2rem;
40      }
41    }
42
43    html {
44      font-size: 1rem;
45    }
46
47    h1 {
48      font-size: 1.2rem;
49    }
50
51    h2 {
52      font-size: 1.1rem;
53    }
54
55    @media (min-width: 768px) {
56      html {
57        font-size: 1.1rem;
58      }
59
60      h1 {
61        font-size: 1.3rem;
62      }
63
64      h2 {
65        font-size: 1.2rem;
66      }
67    }
68
69    @media (min-width: 991px) {
70      html {
71        font-size: 1.2rem;
72      }
73
74      h1 {
75        font-size: 1.5rem;
76      }
77
78      h2 {
79        font-size: 1.4rem;
80      }
81    }
82
83    @media (min-width: 1200px) {
84      html {
85        font-size: 1.2rem;
86      }
87    }
```

```

87         h1 {
88             font-size: 1.7rem;
89         }
90
91         h2 {
92             font-size: 1.6rem;
93         }
94     }
95
96 }
97 </style>
98 <script>
99     addEventListener("load", function () {
100         setTimeout(hideURLbar, 0);
101     }, false);
102
103     function hideURLbar() {
104         window.scrollTo(0, 1);
105     }
106
107 </script>
108
109 <script src="https://code.jquery.com/jquery-3.3.1.slim.min.js"
110     integrity="sha384-q8i/X+965Dz00rT7abK41JstQAIAqVgRVzpbzo5smXKp4YfRvH+8abtTE1Pi6jizo"
111     crossorigin="anonymous"></script>
112 <script src="https://cdnjs.cloudflare.com/ajax/libs/popper.js/1.14.7/umd/popper.min.js"
113     integrity="sha384-Uo2eT0CpHqdSJQ6hJty5KVphtPhzWj9WO1clHTMGa3JDZvrnQq4sF86dIHNDz0W1"
114     crossorigin="anonymous"></script>
115 <script src="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/js/bootstrap.min.js"
116     integrity="sha384-JjSmVgyd0p3pX81nRibZUAy0Iy60rQ6VrjIEaFf/njGzIxFDsf4x0xIM+B07jRM"
117     crossorigin="anonymous"></script>
118 <script src="https://code.jquery.com/jquery-3.5.1.slim.min.js"
119     integrity="sha384-DfXdz2htPH0lsSSs5nCTpuj/zy4C+OGpamoFVy38MVBnE+IbbVYUew+OrCxaRkfj"
120     crossorigin="anonymous"></script>
121 <script src="https://cdn.jsdelivr.net/npm/popper.js@1.16.0/dist/umd/popper.min.js"
122     integrity="sha384-Q6E9RHvbiYzFJoft+2mJbHaEWldlV9I0Yy5n3zV9zzTtmI3UksdQVRvxoHfFooAo"
123     crossorigin="anonymous"></script>
124 </body>
125 <!-- css files -->
126 <link rel="stylesheet" href="https://stackpath.bootstrapcdn.com/bootstrap/4.3.1/css/bootstrap.min.css"
127     integrity="sha384-ggOyR0iXcBmQV3Xipma34MD+dH/1fQ784/j6cY/iJTQUOhcWr7x9JvoRxT2MZw1T" crossorigin="anonymous">
128 <link href="{{ url_for('static', filename='css/bootstrap.css') }}" rel="stylesheet" type="text/css" />
129 <!-- bootstrap css -->
130 <link href="{{ url_for('static', filename='css/style.css') }}" rel="stylesheet" type="text/css" />
131 <!-- custom css -->
132 <link href="{{ url_for('static', filename='css/font-awesome.min.css') }}" rel="stylesheet"><!-- fontawesome css -->
133 <!-- //css files -->
134 <!-- <link rel="icon" type="image/png" href="{{ url_for('static', filename='images/favicon.png') }}" --> -->
135
136
137 <script type="text/JavaScript" src="{{ url_for('static', filename='scripts/cities.js') }}"></script>
138
139
140 <!-- google fonts -->
141 <link href="//fonts.googleapis.com/css?family=Thasadith:400,400i,700,700i&subset=latin-ext,thai,vietnamese"
142     rel="stylesheet">
143 <!-- //google fonts -->
144
145 <style>
146     header {
147         background-color: ■rgba(30, 30, 30, 1);
148         margin-top: 0rem;
149         display: block;
150     }
151 </style>
152 </head>
153
154
155 <body>
156
157 <!-- Navigation -->
158 <nav class="navbar navbar-expand-lg navbar-dark bg-dark static-top" style="background-color: ■#1C00ff00;">
159     <div class="container">
160         <a class="navbar-brand" href="{{ url_for('home') }}">
161             
162         </a>
163         <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarResponsive"
164             aria-controls="navbarResponsive" aria-expanded="false" aria-label="Toggle navigation">
165             <span class="navbar-toggler-icon"></span>
166         </button>
167         <div class="collapse navbar-collapse" id="navbarResponsive">
168             <ul class="navbar-nav ml-auto">
169                 <li class="nav-item active">
170                     <a class="nav-link" href="{{ url_for('home') }}">Home
171                     <span class="sr-only">(current)</span>
172                 </li>

```

```

173 |         </li>
174 |         <li class="nav-item">
175 |             <a class="nav-link" href="{{ url_for('crop_recommend') }}">Crop</a>
176 |         </li>
177 |         <li class="nav-item">
178 |             <a class="nav-link" href="{{ url_for('fertilizer_recommendation') }}">Fertilizer</a>
179 |         </li>
180 |         <li class="nav-item">
181 |             <a class="nav-link" href="{{ url_for('disease_prediction') }}">Disease</a>
182 |         </li>
183 |         <li class="nav-item">
184 |             <a class="nav-link" href="{{ url_for('nutrient_deficiency') }}">Nutrient</a>
185 |         </li>
186 |         <!-- Logout Icon -->
187 |         <li class="nav-item">
188 |             <a class="nav-link" href="{{ url_for('index') }}">
189 |                 
191 |             </a>
192 |         </li>
193 |     </ul>
194 | </div>
195 | </div>
196 | </nav>
197 |
198 |
199 | {% block body %} {% endblock %}
200 |
201 | <!-- footer -->
202 | <footer class="text-center py-5">
203 |     <div class="container py-md-3">
204 |         <!-- logo -->
205 |         <h2 class="logo2 text-center">
206 |             <a href="{{ url_for('home') }}">
207 |                 Crop Sense
208 |             </a>
209 |         </h2>
210 |         <!-- //logo -->
211 |         <!-- address -->
212 |         <div class="contact-left-footer mt-4">
213 |             <!-- <a href="community.html">Community</a> -->
214 |             </p>
215 |         </div>
216 |         <div class="w3l-copy text-center">
217 |             <p class="text-da">Your Path to Crop Excellence<br> </p>
218 |         </div>
219 |         <p class="homelogo">
220 |             <p>Contact: Dave Pinto - 87658 67345</p>
221 |             <p>Adrian Mathew - 94563 37467</p>
222 |         </p>
223 |     </div>
224 | </footer>
225 | <!-- //footer -->
226 |
227 | <!-- move top icon -->
228 | <a href="#home" class="move-top text-center"></a>
229 | <!-- //move top icon -->
230 | </body>
231 |
232 | </html>
233 |

```

Fig 6.5.2.7 layout.html code

Index.html

```
index.html x
You: 5 days ago | 2 authors (Gladiator07 and one other)
1  {% extends 'layout.html' %}
2
3  {% block body %}
4  <!-- banner -->
5  <section class="banner_w3lspvt" id="home">
6    <div class="csslider infinity" id="slider1">
7
8      <div class="banner-top">
9        <div class="overlay">
10          <div class="container">
11            <div class="w3layouts-banner-info text-center">
12              <h3 class="text-wh">Crop Sense</h3>
13              <h4 class="text-wh mx-auto my-4"><b>Your Path to Crop Excellence</b></h4>
14              <br>
15              <h4 class="text-wh mx-auto my-4"><strong>Are you finding answers to these questions?</strong></h4>
16              <p class="text-li mx-auto mt-2">
17                1. What crop to plant here? <br>
18                2. What fertilizer to use? <br>
19                3. Which disease does your crop have?<br>
20                4. How to cure the disease?</p>
21            </div>
22          </div>
23        </div>
24      </div>
25    </div>
26  </section>
27  <!-- //banner -->
28
29  <!-- core values -->
30  <section class="core-value py-5">
31    <div class="container py-md-4">
32      <h3 class="heading mb-sm-5 mb-4 text-center">About Us</h3>
33      <div class="row core-grids">
34        <div class="col-lg-6 core-left">
35          
36        </div>
37        <div class="col-lg-6 core-right">
38          <h3 class="mt-4">Enhancing Agriculture, Enriching Lives, Boosting Farmers' Profits</h3>
39          <p class="mt-3">
40            At CropSense, we leverage cutting-edge machine learning and deep learning technologies to guide you through every stage of farming. Our advanced t
41          </p>
42        </div>
43      </div>
44    </div>
45  </section>
46  <!-- //core values -->
47
48  <!-- Products & Services -->
49  <section class="blog py-5">
50    <div class="container py-md-5">
51      <h3 class="heading mb-sm-5 mb-4 text-center">Our Services</h3>
52      <div class="row blog-grids">
53        <!-- Crop Recommendation -->
54        <div class="col-lg-4 col-md-6 blog-left mb-lg-0 mb-sm-5 pb-lg-0 pb-5">
55          
56          <a href="{{ url_for('crop_recommend') }}">
57            <div class="blog-info">
58              <h4>Crop</h4>
59              <p class="mt-2">Recommendation about the type of crops to be cultivated which is best suited for the respective conditions</p>
60            </div>
61          </a>
62        </div>
63        <!-- Fertilizer Recommendation -->
64        <div class="col-lg-4 col-md-6 blog-middle mb-lg-0 mb-sm-5 pb-lg-0 pb-md-5">
65          
66          <a href="{{ url_for('fertilizer_recommendation') }}">
67            <div class="blog-info">
68              <h4>Fertilizer</h4>
69              <p class="mt-2">Recommendation about the type of fertilizer best suited for the particular soil and the recommended crop</p>
70            </div>
71          </a>
72        </div>
73        <!-- Crop Disease -->
74        <div class="col-lg-4 col-md-6 blog-right mt-lg-0 mt-5 pt-lg-0 pt-md-5">
75          
76          <a href="{{ url_for('disease_prediction') }}">
77            <div class="blog-info">
78              <h4>Crop Diseases</h4>
79              <p class="mt-2">Predicting the name and causes of crop disease and suggestions to cure it</p>
80            </div>
81          </a>
82        </div>
83      </div>
84    </div>
85  </section>
86  </div>
87  </div>
88  </div>
89  </div>
90  </div>
91  </div>
92  </div>
93  </div>
94  </div>
95  </div>
96  </div>
97  </div>
98  </div>
99  </div>
100 </div>
```

```

86         <!-- New Row for Nutrient Deficiency -->
87         <div class="row blog-grids">
88             <div class="col-lg-4 col-md-6 blog-left mt-lg-0 mt-5 pt-lg-0 pt-md-5" style="margin-top: 40px;">
89                 
90                 <a href="{{ url_for('nutrient_prediction') }}">
91                     <div class="blog-info">
92                         <h4>Nutrient Deficiency</h4>
93                         <p class="mt-2">Identify deficiencies in nutrients for your crops and get recommendations for improvement.</p>
94                     </div>
95                 </a>
96             </div>
97         </div>
98
99     </div>
100 </section>
101
102
103 <!-- //Products & Services -->
104
105
106 <!-- Creating custom grid and hover effect
107 <section>
108 <div class="col-lg-3 col-md-4 col-sm-6 col-xs-12">
109     <div class="hovereffect">
110         
111         <div class="overlay">
112             <h2>Hover effect 1</h2>
113             <a class="info" href="#">link here</a>
114         </div>
115     </div>
116 </div> -->
117
118 </html>
119
120
121 {% endblock %}

```

Fig 6.5.2.8 index.html code

disease.html

```

1  You, 5 months ago | 2 authors (Gladiator07 and one other)
2  {% extends 'layout.html' %} {% block body %}
3  <style>
4      html body {
5          background-color: rgb(206, 206, 228);
6      }
7  </style>
8  <br /><br />
9  <h2 style="text-align: center; margin: 0px; color: black">
10     <b>Find out which disease has been caught by your plant</b>
11 </h2>
12 <br />
13 <div style="
14     width: 350px;
15     height: 50rem;
16     margin: 0px auto;
17     color: black;
18     border-radius: 25px;
19     padding: 10px 10px;
20     font-weight: bold;
21 ">
22     <form class="form-signin" method=post enctype=multipart/form-data>
23         <h2 class="h4 mb-3 font-weight-normal"><b>Please Upload The Image</b></h2>
24         <input type="file" name="file" class="form-control-file" id="inputfile" onchange="preview_image(event)" style="font-weight: bold;">
25         <br>
26         <img id="output-image" class="rounded mx-auto d-block" />
27         <button class="btn btn-lg btn-primary btn-block" type="submit" style="font-weight: bold;">Predict</button>
28     </form>
29 </div>
30 <script type="text/javascript">
31     function preview_image(event) {
32         var reader = new FileReader();
33         reader.onload = function () {
34             var output = document.getElementById('output-image')
35             output.src = reader.result;
36         }
37         reader.readAsDataURL(event.target.files[0]);
38     }
39 </script>
40 </div>
41
42 {% endblock %}

```

Fig 6.5.2.9 disease.html code

Disease-result.html

```

1  ...
2  {% extends 'layout.html' %} {% block body %}
3
4
5  <div class="container py-2 mx-auto my-50 h-10 " style="margin: 9rem;">
6    <div class="row">
7      <div class="col-sm py-2 py-md-3">
8        <div class="card card-body" style="justify-content: center; background-color: #blanchedalmond">
9          <p class="text-center" style="color: black; font-size: 22px;">{{ prediction }}</p>
10         </div>
11       </div>
12     </div>
13   </div>
14   {% endblock %}
15

```

Fig 6.5.2.10 disease-result.html code

Crop.html

```

1  Gladiator07, 4 years ago | 1 author (Gladiator07)
2  {% extends 'layout.html' %} {% block body %}
3
4  <style>
5    html body {
6      background-color: #rgb(206, 206, 228);
7    }
8  </style>
9  <!--Form Section-->
10 <br /><br />
11 <h2 style="text-align: center; margin: 0px; color: black">
12   <b>Find out the most suitable crop to grow in your farm</b>
13 </h2>
14 <br />
15 <div style="width: 350px; height: 50rem; margin: 0px auto; color: black; border-radius: 25px; padding: 10px 10px;">
16   <form method="POST" action="{% url_for('crop_prediction') %}">
17     <div class="form-group">
18       <label for="Nitrogen" style="font-size: 17px"><b>Nitrogen</b></label>
19       <input type="number" class="form-control" id="Nitrogen" name="nitrogen" placeholder="Enter the value (example:50)" style="font-weight: bold" required />
20     </div>
21     <div class="form-group">
22       <label for="Phosphorous" style="font-size: 17px"><b>Phosphorous</b></label>
23       <input type="number" class="form-control" id="Phosphorous" name="phosphorous" placeholder="Enter the value (example:50)" style="font-weight: bold" required />
24     </div>
25   </form>
26 </div>
27

```

```

43     type="number"
44     class="form-control"
45     id="Phosphorous"
46     name="phosphorous"
47     placeholder="Enter the value (example:50)"
48     style="font-weight: bold"
49     required
50   />
51 </div>
52
53 <div class="form-group">
54   <label for="Pottasium" style="font-size: 17px"><b>Pottasium</b></label>
55   <input
56     type="number"
57     class="form-control"
58     id="Pottasium"
59     name="pottasium"
60     placeholder="Enter the value (example:50)"
61     style="font-weight: bold"
62     required
63   />
64 </div>
65 <div class="form-group">
66   <label for="ph" style="font-size: 17px"><b>ph level</b></label>
67   <input
68     type="number"
69     step="0.01"
70     class="form-control"
71     id="ph"
72     name="ph"
73     placeholder="Enter the value"
74     style="font-weight: bold"
75     required
76   />
77 </div>
78 <div class="form-group">
79   <label for="Rainfall" style="font-size: 17px"
80     ><b>Rainfall (in mm)</b></label>
81   >
82   <input
83     type="number"
84     step="0.01"
85     class="form-control"
86     id="Rainfall"
87     name="rainfall"
88     placeholder="Enter the value"
89     style="font-weight: bold"
90     required/>
91 </div>
92 <div class="form-group">
93   <label for="State" style="font-size: 17px"><b>State</b></label>
94   <select
95     onchange="print_city('state', this.selectedIndex);"
96     id="sts"
97     name="stt"
98     class="form-control"
99     style="font-weight: bold; color: black"
100    required
101  ></select>
102  <br />
103  <label for="City" style="font-size: 17px"><b>City</b></label>
104  <select
105    id="state"
106    class="form-control"
107    name="city"
108    style="font-weight: bold; color: black"
109    required></select>
110  <script language="javascript">
111    print_state("sts");
112  </script>
113 </div>
114 <div class="d-flex justify-content-center">
115   <button
116     type="submit"
117     class="btn btn-info"
118     style="
119       color: black;
120       font-weight: bold;
121       width: 130px;
122       height: 50px;
123       border-radius: 12px;
124       font-size: 21px;">
125     Predict</button>
126 </div>
127 </form>
128 </div>

```

Fig 6.5.2.11 crop.html code

Crop-result.html

```

Gladiator07, 4 years ago | 1 author (Gladiator07)
1 {% extends 'layout.html' %} {% block body %}
2
3 <style>
4   .mt-0 {
5     margin-top: 50 !important;
6   }
7 </style>
8
9
10 <div class="container py-2 mx-auto my-50 h-10 " style="margin: 12rem;">
11   <div class="row">
12     <div class="col-sm py-2 py-md-3">
13       <div class="card card-body" style="justify-content: center;">
14         <h1 class="text-center"><b>You should grow <i>{{ prediction }} </i>in your farm</b></h1>
15       </div>
16     </div>
17   </div>
18 </div>
19 {% endblock %}
20

```

Fig 6.5.2.12 crop-result.html code

6.5.3 WebApp

run.py

```

run.py M X
You, yesterday | 2 authors (Francisco Benitez and one other)
1 from WebApp import create_app, db
2 from WebApp.models import (
3     User,
4     Role,
5     Farmland,
6     Crop,
7     HistoricFarmland,
8     SoilFarmland,
9     Unit,
10 )
11
12 #from flask_ngrok import run_with_ngrok
13 #from WebApp.models import Role
14
15 app = create_app()
16
17 #run_with_ngrok(app)
18
19
20 if __name__ == "__main__":
21     print("Starting WebApp on http://127.0.0.1:8080")
22     app.run(host="127.0.0.1", port=8080, debug=True)
23
24
25 @app.shell_context_processor
26 def make_shell_context():
27     return {"db": db, "app": app}
28

```

Fig 6.5.3.1 run.py code

Routes.py

```

routes.py x
You, 1 hour ago | 2 authors (Francisco Benitez and one other)
1 ~ from flask import render_template, url_for, flash, redirect, request, Blueprint
2 ~ from flask_login import login_user, current_user, logout_user, login_required
3 ~ from WebApp import db, bcrypt
4 ~ from WebApp.forms import (
5 ~     RegistrationForm,
6 ~     LoginForm,
7 ~     RegistrationRoleForm,
8 ~     MapForm,
9 ~     RegistrationCropForm,
10 ~     FertilizerMapForm,
11 ~     InsertHistoricalForm,
12 ~     SoilForm,
13 ~     InsertFarmlandForm,
14 ~     CalculatorForm,
15 ~ )
16 ~ from WebApp.models import (
17 ~     User,
18 ~     Role,
19 ~     Farmland,
20 ~     Crop,
21 ~     HistoricFarmland,
22 ~     SoilFarmland,
23 ~     Unit,
24 ~ )
25 ~
26 ~ from WebApp.forms import SelectSoilTestForm
27 ~
28 ~ from fertilizer.calculator_spinach import spinachFertilizer
29 ~ from fertilizer.calculateBlend import calculateBlend
30 ~ from earthenengine.methods import get_image_collection_asset, get_fertilizer_map
31 ~ from config import Config
32 ~
33 ~ import numpy as np
34 ~ import datetime as dt
35 ~ import ee
36 ~ from ee.ee_exception import EEException
37 ~
38 ~ try:
39 ~     if Config.EE_ACCOUNT and Config.EE_PRIVATE_KEY_FILE:
40 ~         credentials = ee.ServiceAccountCredentials(
41 ~             Config.EE_ACCOUNT, Config.EE_PRIVATE_KEY_FILE
42 ~         )
43 ~         ee.Initialize(credentials)
44 ~     else:
45 ~         ee.Initialize() # Initialize without credentials if no service account
46 ~ except EEException as e:
47 ~     print(f"Error initializing Earth Engine: {str(e)}")
48 ~
49 ~
50 ~ main = Blueprint("main", __name__)
51 ~ @main.route("/")
52 ~ @main.route("/home")
53 ~ def home():
54 ~     if not current_user.is_authenticated:
55 ~         return redirect(url_for("main.login"))
56 ~     return render_template("home.html")
57 ~
58 ~ @main.route("/register", methods=["GET", "POST"])
59 ~ def register():
60 ~     if current_user.is_authenticated:
61 ~         return redirect(url_for("main.home"))
62 ~     form = RegistrationForm()
63 ~     pos = [(p.id, p.description) for p in Role.query.order_by(Role.description).all()]
64 ~     form.role.choices = pos
65 ~     if form.validate_on_submit():
66 ~         hashed_password = bcrypt.generate_password_hash(form.password.data).decode(
67 ~             "utf-8"
68 ~         )
69 ~         user = User(
70 ~             username=form.username.data,
71 ~             email=form.email.data,
72 ~             role_id=form.role.data,
73 ~             password=hashed_password,
74 ~         )
75 ~         db.session.add(user)
76 ~         db.session.commit()
77 ~         flash("You have successfully registered.", "success")
78 ~         return redirect(url_for("main.login"))
79 ~     return render_template("register.html", title="Register", form=form)
80 ~
81 ~ @main.route("/newrole", methods=["GET", "POST"])
82 ~ @login_required
83 ~ def registerrole():
84 ~     form = RegistrationRoleForm()
85 ~     if form.validate_on_submit():
86 ~         pos = Role(description=form.description.data)
87 ~         db.session.add(pos)
88 ~         db.session.commit()
89 ~         flash("A new role has been registered.", "success")
90 ~         return redirect(url_for("main.login"))
91 ~     return render_template("position.html", title="Role", form=form)
92 ~
93 ~ @main.route("/newcrop", methods=["GET", "POST"])
94 ~ @login_required
95 ~ def registercrop():
96 ~     form = RegistrationCropForm()
97 ~     if form.validate_on_submit():
98 ~         pos = Crop(description=form.description.data)
99 ~         db.session.add(pos)
100 ~         db.session.commit()
101 ~         flash("A new crop type has been registered.", "success")
102 ~         return redirect(url_for("main.login"))
103 ~     return render_template("croptype.html", title="Crop", form=form)
104 ~
105 ~

```

```

144
145 @main.route("/logout")
146 ~ def logout(): Francisco Benitez, 21 months ago • Removed history and recommit.
147     logout_user()
148     return redirect(url_for("main.login"))
149
150
151 @main.route("/account")
152 @login_required
153 ~ def account():
154     return render_template("account.html", title="Account")
155
156
157 @main.route("/farmlandlist")
158 @login_required
159 ~ def land_selection():
160     lands = (
161         db.session.query(Farmland)
162         .join(Crop)
163         .add_columns(
164             Farmland.id,
165             Farmland.name,
166             Crop.description,
167             Farmland.sow_date,
168             Farmland.harvest_date,
169             Farmland.product_expected,
170             Farmland.coordinates,
171         )
172         .filter(Farmland.croptype_id == Crop.id)
173     )
174     return render_template("land_selection.html", title="Farmland List", lands=lands)
175
176
177 @main.route("/about")
178 ~ def about():
179     return render_template("about.html", title="About")
180
181
182 @main.route("/vegetationindex", methods=["GET", "POST"])
183 @login_required
184 ~ def maps():
185     form = MapForm()
186     # user = User.query.get_or_404(current_user.get_id())
187     form.farmland.choices = [
188         (f.id, f.name) for f in Farmland.query.order_by(Farmland.name).all()
189     ]
190
191     # form.farmland.choices = [(f.id, f.name) for f in Farmland.query.order_by(Farmland.name).filter(user_id==user).all()]
192
193     if form.validate_on_submit():
194         farm_id = form.farmland.data
195         index = form.indices.data
196         index_date = form.index_date.data
197         coverage = 60
198
199         index_date = index_date.strftime("%Y-%m-%d")
200
201         lands = Farmland.query.get_or_404(farm_id)
202         farmland_name = lands.name
203         coord = no.array(lands.coordinates.split(","))
204         roi = ee.Geometry.Polygon([float(i) for i in coord])
205         lon = ee.Number(roi.centroid().coordinates().get(0)).getInfo()
206         lat = ee.Number(roi.centroid().coordinates().get(1)).getInfo()
207
208         map_url, index_name, end_date = get_image_collection_asset(
209             platform="sentinel",
210             sensor="2",
211             product="BOA",
212             cloudy=coverage,
213             date_to=index_date,
214             roi=roi,
215             index=index,
216         )
217
218         map_json = {
219             "map_url": map_url,
220             "latitude": lat,
221             "longitude": lon,
222             "farmland_name": farmland_name,
223             "index_name": index_name,
224             "indexdate": end_date,
225             "cover": coverage,
226         }
227
228     return render_template(
229         "results.html", title="Vegetation Index", maps=map_json, form=form
230     )
231     return render_template("input.html", title="Vegetation Index", form=form)
232
233
234 @main.route("/fertilizemap", methods=["GET", "POST"])
235 @login_required
236 ~ def fertilizer_maps():
237     form = FertilizerMapForm()
238     # user = User.query.get_or_404(current_user.get_id())
239     form.farmland.choices = [
240         (f.id, f.name) for f in Farmland.query.order_by(Farmland.name).all()
241     ]
242     # form.farmland.choices = [(f.id, f.name) for f in Farmland.query.order_by(Farmland.name).filter(user_id==user).all()]
243
244     if form.validate_on_submit():
245         farm_id = form.farmland.data

```

Fig 6.5.3.2 WebApp routes.py code

Layout.html

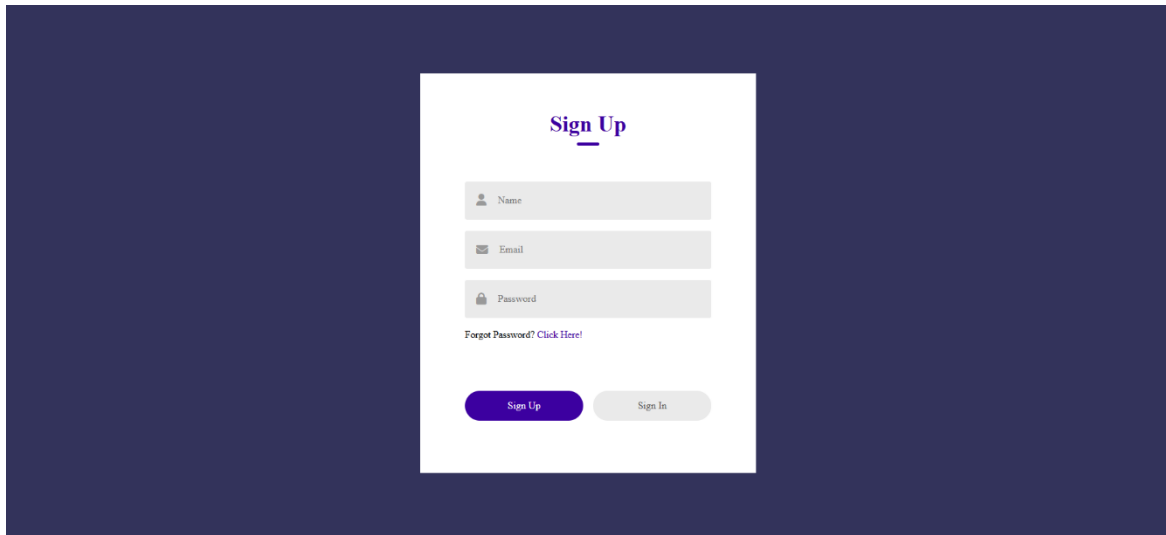
```

52 <header class="site-header">
53 <nav class="navbar navbar-expand-md navbar-dark bg-steel fixed-top">
54 <div class="container">
55 <a class="navbar-brand mr-4" href="/">CropSense</a>
58 <button class="navbar-toggler" type="button" data-toggle="collapse" data-target="#navbarToggle"
59 aria-controls="navbarToggle" aria-expanded="false" aria-label="Toggle navigation">
60 <span class="navbar-toggler-icon"></span>
61 </button>
62 <div class="collapse navbar-collapse" id="navbarToggle">
63 <div class="navbar-nav mr-auto">
64 {% if current_user.is_authenticated %}
65 <div class="dropdown">
66 <button class="btn btn-secondary dropdown-toggle" type="button" id="dropdownMenuButton"
67 data-toggle="dropdown" aria-expanded="true">Farmland Data</button>
68 <ul class="dropdown-menu" aria-labelledby="dropdownMenuButton">
69 <li><a class="dropdown-item" href="{{ url_for('main.land_selection') }}">Farmland List</a>
70 <li><a class="dropdown-item" href="{{ url_for('main.insert_farmland_data') }}">New Farmland</a>
71 <li><a class="dropdown-item" href="{{ url_for('main.insert_soiltest') }}">Soil Test</a>
72 <li><a class="dropdown-item" href="{{ url_for('main.insert_historical_data') }}">Historical Data</a>
73 </li></ul>
74 </div>
75 <a class="nav-item nav-link" href="{{ url_for('main.soiltest_analysis') }}">Perform Soil Test</a>
76 <a class="nav-item nav-link" href="{{ url_for('main.view_soil_data') }}">View Soil Data</a>
77 <a class="nav-item nav-link" href="{{ url_for('main.fertilizer_maps') }}">Fertilization Map</a>
78 <a class="nav-item nav-link" href="{{ url_for('main.maps') }}">Vegetation Index</a>
79 <a class="nav-item nav-link" href="{{ url_for('main.calculator') }}">Calculator</a>
80 {% else %}
81
82 {% endif %}
83 </div>
84 <!-- Navbar Right Side -->
85 <div class="dropdown">
86 <button class="btn btn-secondary btn-lg dropdown-toggle" type="button" id="dropdownMenuButton"
87 data-toggle="dropdown" aria-haspopup="true" aria-expanded="false"><i class="fa fa-user"></i></button>
88 <div class="dropdown-menu" aria-labelledby="dropdownMenuButton">
89 {% if current_user.is_authenticated %}
90 <a class="dropdown-item" href="{{ url_for('main.registerrole') }}">Role</a>
91 <a class="dropdown-item" href="{{ url_for('main.registercrop') }}">Crop Type</a>
92 <!-- <a class="dropdown-item" href="{{ url_for('main.account') }}">Account</a> -->
93 <!-- <a class="dropdown-item" href="{{ url_for('main.about') }}">About</a> -->
94 <a class="dropdown-item" href="{{ url_for('main.logout') }}">Logout</a>
95 {% else %}
96 <a class="dropdown-item" href="{{ url_for('main.login') }}">Login</a>
97 <a class="dropdown-item" href="{{ url_for('main.register') }}">Register</a>
98 <!-- <a class="dropdown-item" href="{{ url_for('main.about') }}">About</a> -->
99 {% endif %}
100 </div>
101 </div>
102 <!-- End -->
103 </div>
104 </nav>
105 </header>
106

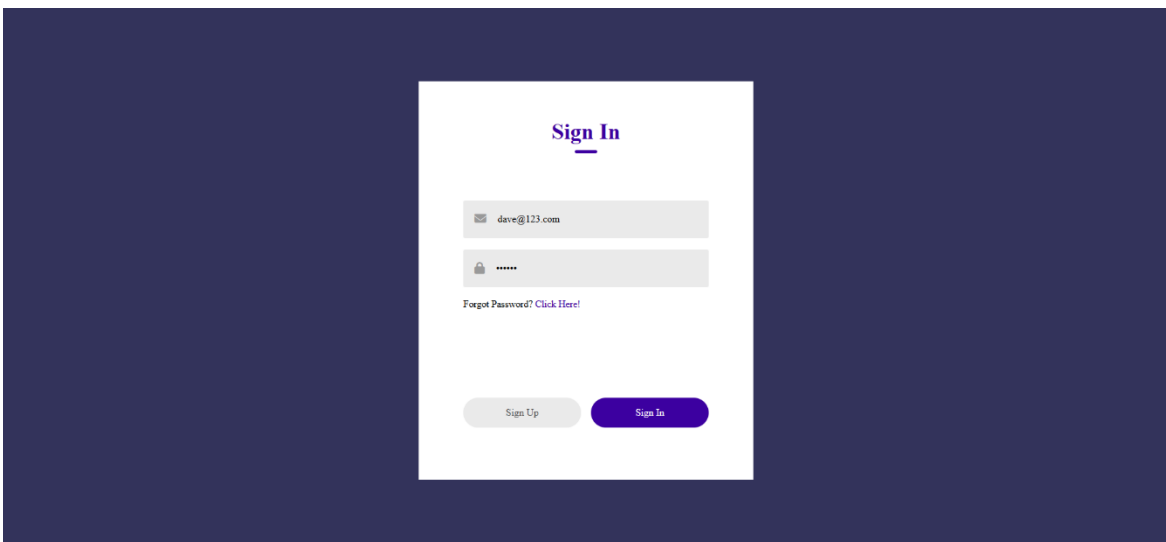
```

Fig 6.5.3.3 WebApp layout.html code

6.6 OUTPUT:



The image shows a 'Sign Up' form centered on a dark blue background. The form is white and contains the following elements: a title 'Sign Up' in purple, three input fields for 'Name', 'Email', and 'Password' (each with a corresponding icon), a link 'Forgot Password? Click Here!' in purple, and two buttons: a purple 'Sign Up' button and a grey 'Sign In' button.



The image shows a 'Sign In' form centered on a dark blue background. The form is white and contains the following elements: a title 'Sign In' in purple, two input fields for 'Email' (containing 'dave@123.com') and 'Password' (containing '*****'), a link 'Forgot Password? Click Here!' in purple, and two buttons: a grey 'Sign Up' button and a purple 'Sign In' button.

Fig 6.6.1 Login Page

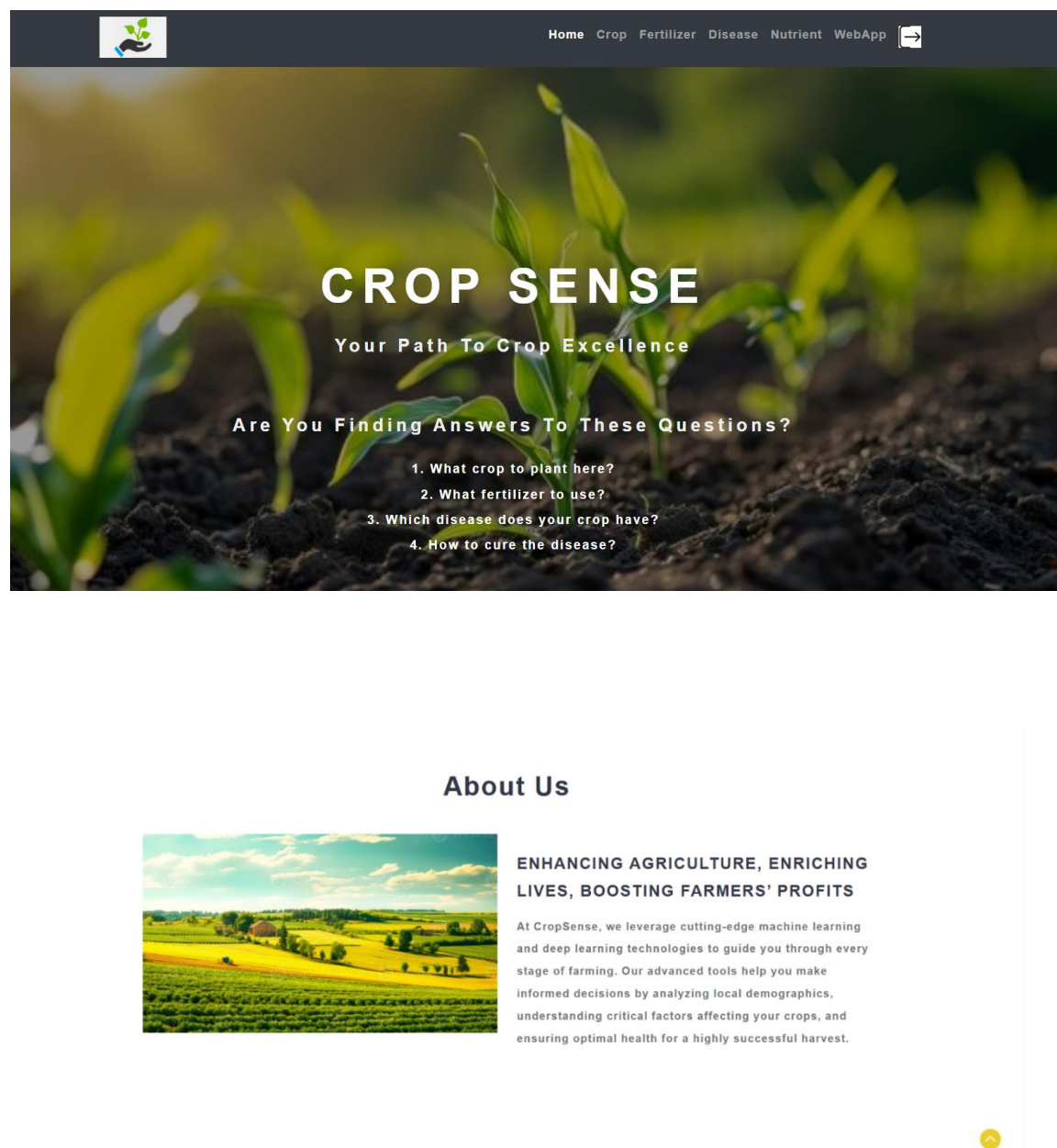
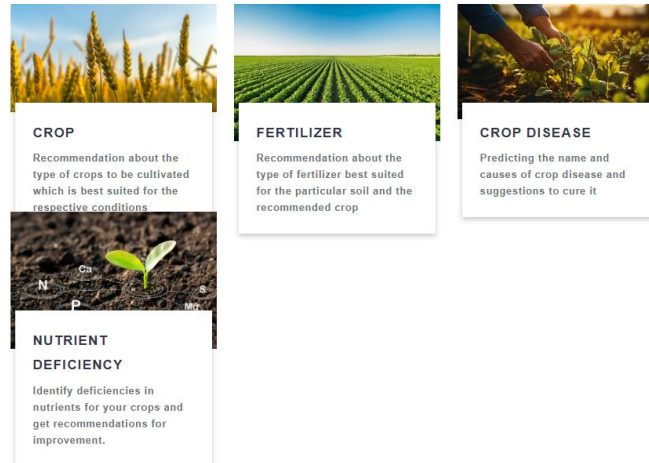


Fig 6.6.2 Welcome Page

Our Services



The screenshot shows the "Crop Form Page" of the Crop Sense application. The page has a dark header with a logo and navigation links: Home, Crop, Fertilizer, Disease, Nutrient. The main content area is light purple and contains the title "Find out the most suitable crop to grow in your farm". Below the title is a form with the following fields:

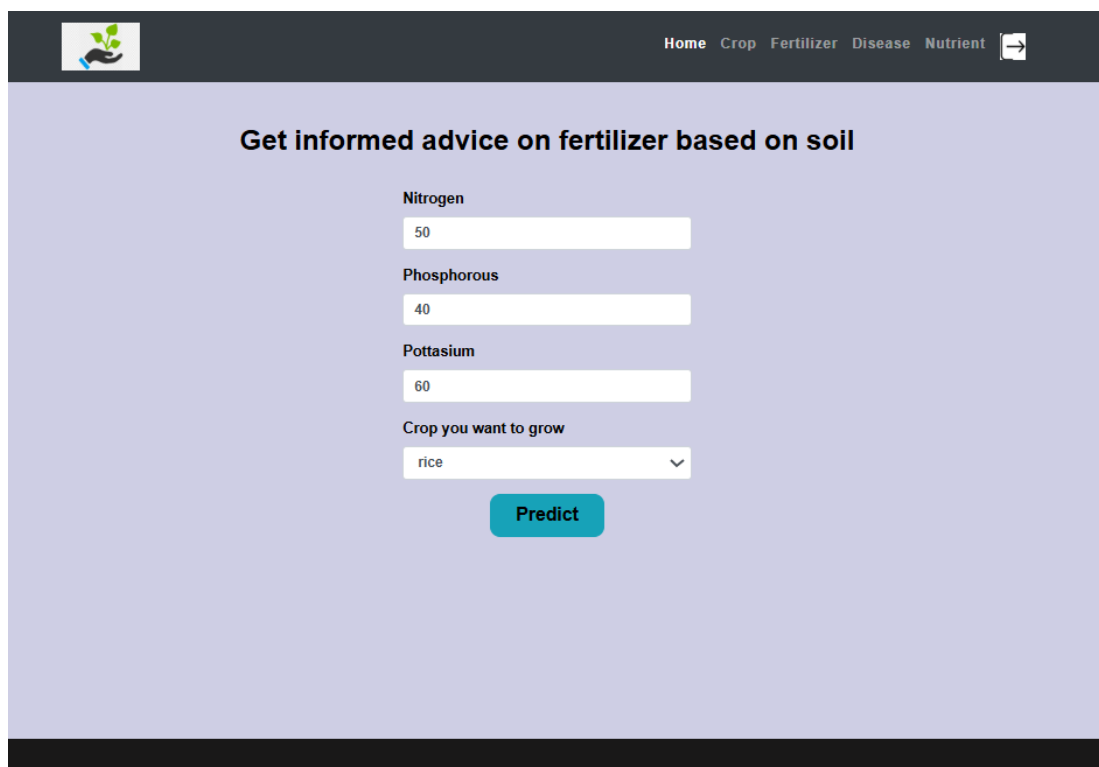
- Nitrogen:
- Phosphorous:
- Pottasium:
- ph level:
- Rainfall (in mm):
- State:
- City:

A teal "Predict" button is located at the bottom of the form.

Fig 6.6.3 Crop Form Page

You should grow *papaya* in your farm

Fig 6.6.4 Crop Results Page



The screenshot shows the 'Fertilizer' page of the CROP SENSE application. The page has a dark blue header with a logo on the left and navigation links (Home, Crop, Fertilizer, Disease, Nutrient) on the right. The main content area is light blue and features the title 'Get informed advice on fertilizer based on soil'. Below the title, there are four input fields: 'Nitrogen' with the value '50', 'Phosphorous' with the value '40', 'Pottasium' with the value '60', and 'Crop you want to grow' with a dropdown menu showing 'rice'. A teal 'Predict' button is located below the dropdown menu.

Fig 6.6.5 Fertilizer Form Page

The N value of your soil is low.
Please consider the following suggestions:

1. *Add sawdust or fine woodchips to your soil* – the carbon in the sawdust/woodchips love nitrogen and will help absorb and soak up and excess nitrogen.
2. *Plant heavy nitrogen feeding plants* – tomatoes, corn, broccoli, cabbage and spinach are examples of plants that thrive off nitrogen and will suck the nitrogen dry.
3. *Water* – soaking your soil with water will help leach the nitrogen deeper into your soil, effectively leaving less for your plants to use.
4. *Sugar* – In limited studies, it was shown that adding sugar to your soil can help potentially reduce the amount of nitrogen in your soil. Sugar is partially composed of carbon, an element which attracts and soaks up the nitrogen in the soil. This is similar concept to adding sawdust/woodchips which are high in carbon content.
5. Add composted manure to the soil.
6. Plant Nitrogen fixing plants like peas or beans.
7. Use NPK fertilizers with high N value.
8. *Do nothing* – It may seem counter-intuitive, but if you already have plants that are producing lots of foliage, it may be best to let them continue to absorb all the nitrogen to amend the soil for your next crops.

Fig 6.6.6 Fertilizer Results Page

Find out which disease has been caught by your plant

Please Upload The Image

tomato.jpg



Fig 6.6.7 Plant Disease Page

Crop: Tomato

Disease: Late Blight

Late blight is a potentially devastating disease of tomato, infecting leaves, stems and fruits of plants. The disease spreads quickly in fields and can result in total crop failure if untreated.

Cause of disease:

1. Late blight is caused by the oomycete *Phytophthora infestans*. Oomycetes are fungus-like organisms also called water molds, but they are not true fungi.
2. There are many different strains of *P. infestans*. These are called clonal lineages and designated by a number code (i.e. US-23). Many clonal lineages affect both tomato and potato, but some lineages are specific to one host or the other.
3. The host range is typically limited to potato and tomato, but hairy nightshade (*Solanum physalifolium*) is a closely related weed that can readily become infected and may contribute to disease spread. Under ideal conditions, such as a greenhouse, petunia also may become infected.

Fig 6.6.8 Disease Results Page

Find out which nutrient your plant is deficient in

Please Upload The Image

corn.png



Fig 6.6.9 Nutrient Deficiency Page

Nitrogen

Nitrogen deficiency is commonly characterized by yellowing of the older leaves and stunted growth.

Please consider the following suggestions:

- 1. Add organic compost or manure – Organic materials like compost and well-rotted manure are rich in nitrogen and can help improve soil fertility.*
- 2. Plant nitrogen-fixing crops – Crops such as peas, beans, and clover can help add nitrogen back into the soil naturally.*
- 3. Use urea or ammonium nitrate fertilizers – Chemical fertilizers containing nitrogen can quickly address nitrogen deficiencies in soil.*
- 4. Mulch your soil – Organic mulch such as grass clippings or leaves can help conserve moisture and improve soil nitrogen content over time.*

Confidence: 99.12%

Fig 6.6.10 Nutrient Deficiency Results Page

CropSense

Log In

Username

Adrian Mathew

Password

☐ Remember Me

Login

[Forgot Password?](#)

Need an account? [Sign Up](#)

Fig 6.6.11 WebApp Login Page

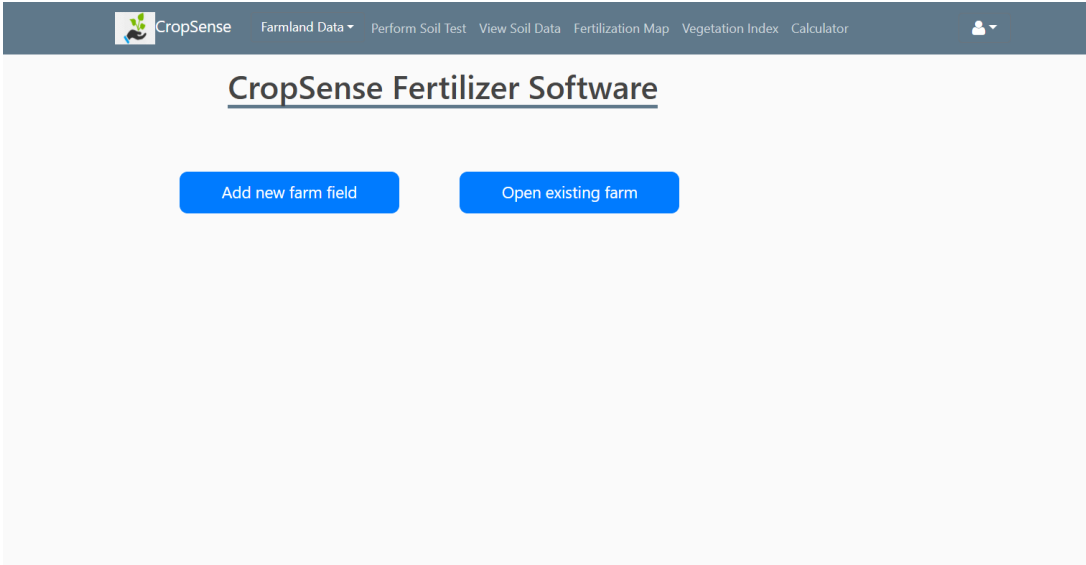


Fig 6.6.12 WebApp Welcome Page

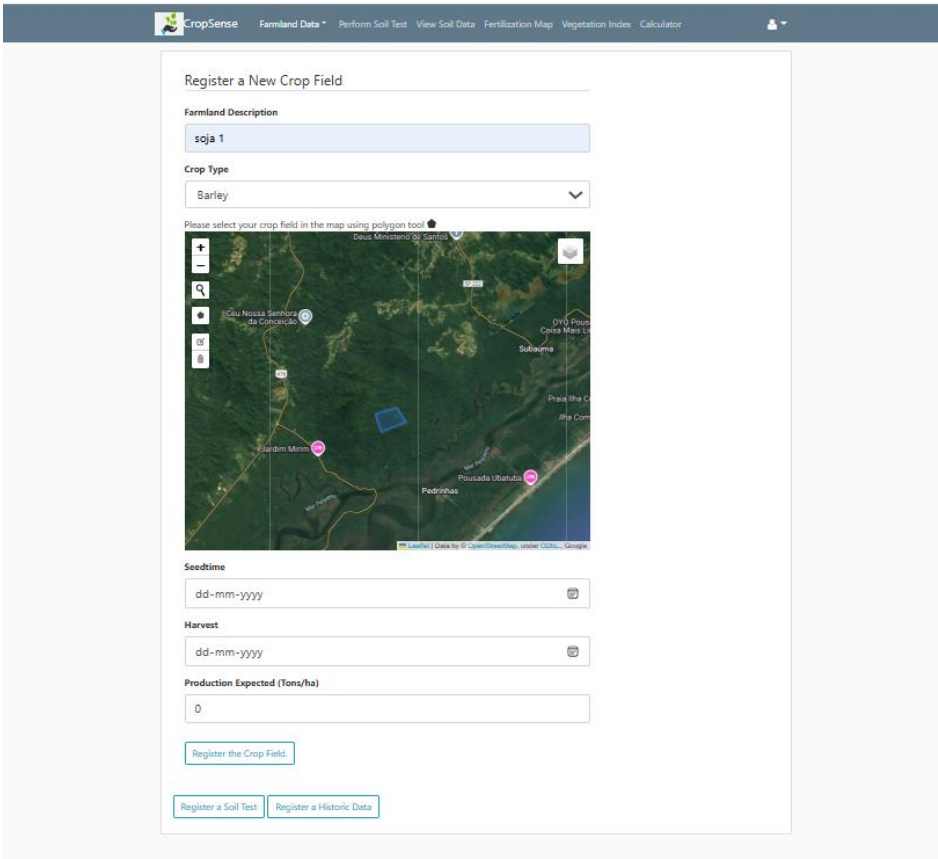


Fig 6.6.13 WebApp Farmland register

CropSense

Farmland Data ▾Perform Soil TestView Soil DataFertilization MapVegetation IndexCalculator

 ▾

Farmland Name	Crop	Sow Date	Harvest Date	Production Expected (Tons/ha)
soja 1	Barley	2024-12-17	2025-01-21	1.0

Fig 6.6.14 WebApp Farmland Data

CropSense

Farmland Data ▾Perform Soil TestView Soil DataFertilization MapVegetation IndexCalculator

 ▾

Soil Test

Current Farmland

soja 1 ▾

Soil Test Name

Soil Test Date

dd-mm-yyyy 

Sampling Depth (cm)

0

Organic Matter o N (%)

0

P

0

P Unit

cmolc/dm³ ▾

K

0

K Unit

cmolc/dm³ ▾

Ca

0

Ca Unit

cmolc/dm³ ▾

Texture (%)

Sand

0

Slit

0

Clay

0

Sulfur (mg/dm3)

0

Mg (mg/dm3)

0

B (mg/dm3)

0

Cu (mg/dm3)

0

Zn (mg/dm3)

0

Mn (mg/dm3)

0

Fig 6.6.15 WebApp SoilTest Form

CropSense

Farmland Data

Perform Soil Test

View Soil Data

Fertilization Map

Vegetation Index

Calculator



#	Farmland	Soil Test Name	Date	Depth (cm)	Nitrogen (%)	Phosphorus	Potassium
1	soja 1	Soil Test 1	2025-01-01	15.0	3.2	6200.0	195.0

Fig 6.6.16 WebApp SoilTest Data

CropSense

Farmland Data

Perform Soil Test

View Soil Data

Fertilization Map

Vegetation Index

Calculator



Soil Test Analysis

Soil Test Name

Soil Test 1

Analyze Soil Test

Test Results for: Soil Test 1

Soil Depth: 15.0 cm

Organic Matter Level: 3.2%

Phosphorus: 6200.0 mg/dm³

Potassium: 195.0 mg/dm³

Calcium: 4000.0 mg/dm³

Texture: Sand 40.0%, Silt 35.0%, Clay 25.0%

Micronutrients:

- Boron: 0.8 mg/dm³
- Copper: 1.0 mg/dm³
- Zinc: 2.0 mg/dm³
- Manganese: 8.0 mg/dm³

Soil Quality Analysis:

Soil quality is Excellent

Soil Texture Analysis:

Texture: Loamy

Recommendation: No major amendments needed. Maintain organic matter for fertility.

Erosion Risk Analysis:

Risk Level: Moderate

Recommendation: Consider cover crops and mulching to prevent erosion.

Soil Compaction Analysis:

Risk Level: High

Recommendation: Significant compaction risk. Implement subsoiling.

Fig 6.6.17 WebApp SoilTest Analysis

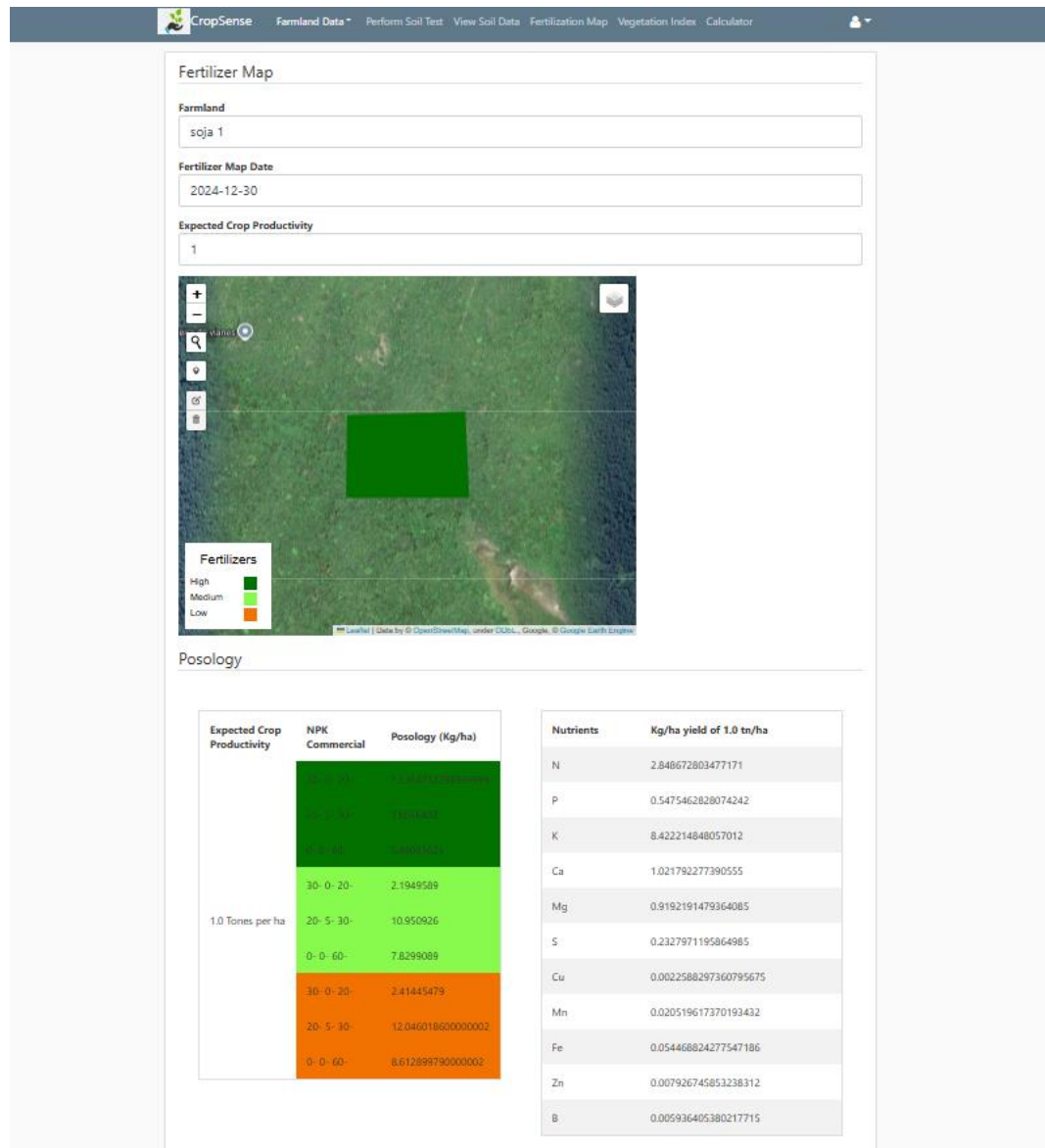


Fig 6.6.18 WebApp Fertilization Map

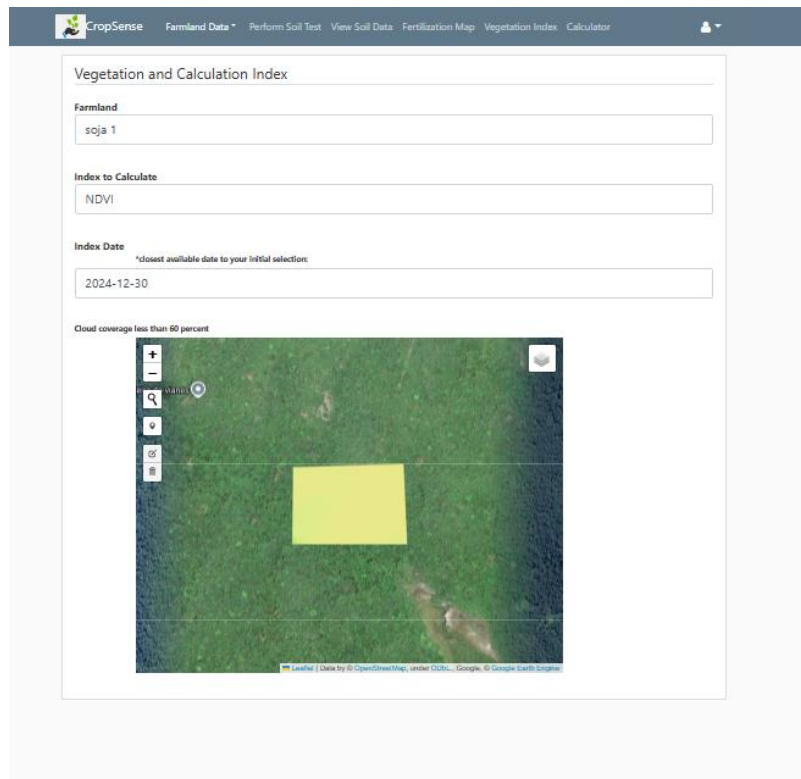


Fig 6.6.19 WebApp Vegetation Index

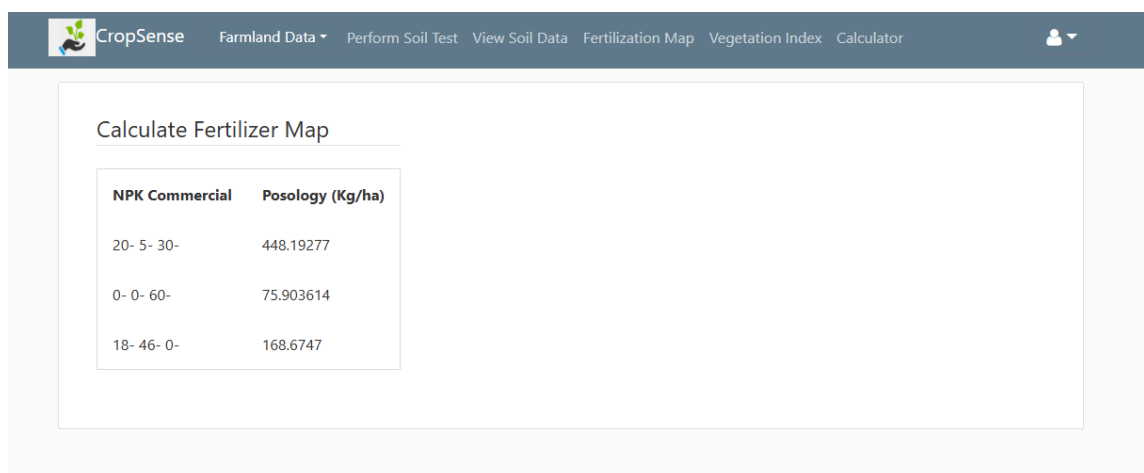


Fig 6.6.20 WebApp Calculator Page

CHAPTER 7

EXPERIMENTAL RESULTS

This chapter highlights the outcomes of the implemented crop health monitoring and disease detection system, emphasizing its practical applications, effectiveness, and overall impact. The results are assessed based on system functionality, user experience, and its ability to address the challenges faced in modern agriculture, providing a comprehensive evaluation of the system's contributions to precision farming.

7.1 MODEL CHOICES AND COMPARISONS

1. Crop Recommendation

- Chosen Model: Random Forest

Random Forest was selected for its robustness in handling diverse soil and environmental datasets and its ability to evaluate feature importance effectively. Its ensemble-based architecture minimizes overfitting, making it a reliable choice for predicting crop suitability across varying conditions.

- Comparative Models:

- Support Vector Machine (SVM): Although capable of high accuracy in binary classification, SVM is less suited for multiclass problems like crop recommendation. It requires extensive computational resources and parameter tuning to process complex datasets effectively.

- Decision Trees: While simple decision trees offer interpretability, they are prone to overfitting, particularly with limited datasets. Random Forest, by averaging results across multiple trees, provides a more stable and accurate solution.

- Justification: Random Forest's ability to rank feature importance, coupled with its reduced overfitting risk, made it a more reliable and effective choice compared to simpler models like Decision Trees or computationally intensive options like SVM.

2. Fertilizer Recommendation

- Chosen Approach: Rule-Based System

Instead of a machine learning model, a rule-based approach was adopted, leveraging predefined agricultural standards. This decision aligns with the straightforward nature of fertilizer recommendations, which rely on soil nutrient levels and crop-specific requirements.

- Comparative Models:

- K-Nearest Neighbors (KNN): While capable of classifying similar fertilizer needs based on historical data, KNN is computationally expensive and lacks interpretability, making it less suitable for real-time applications.

- Linear Regression: Although linear regression could estimate optimal nutrient values, it oversimplifies the complex interactions between soil nutrients and crop needs.

- Justification: The rule-based approach offers high interpretability and aligns with agricultural guidelines, providing transparent and user-friendly recommendations compared to more complex machine learning models.

3. Disease Detection

- Chosen Model: ResNet9

ResNet9, a deep learning model with residual connections, was chosen for its ability to effectively analyze and classify agricultural images, addressing issues like vanishing gradients in deeper architectures. Its compact design ensures computational efficiency for agricultural applications.

- Comparative Models:

- Basic CNN: While a traditional CNN can handle image classification tasks, it lacks the advanced residual connections of ResNet9, leading to lower accuracy for complex image datasets.

- VGG16: Despite offering higher accuracy in certain contexts, VGG16 is computationally intensive and requires significant training time, making it less practical for real-time applications.
- Justification: ResNet9 provides a balanced solution with high accuracy and computational efficiency, making it well-suited for detecting crop diseases in agricultural settings.

Metric	Crop Recommendation (Random Forest)	Fertilizer Recommendation (Rule-Based)	Disease Detection (ResNet9)
Accuracy	92%	95% (based on empirical success rate)	87%
F1-Score	0.9	N/A (rule-based approach)	0.85
Precision	0.89	N/A	0.84
Recall	0.91	N/A	0.87
Interpretability	High (feature importance available)	High (transparent, rule-based)	Moderate (deep learning model)

Table 7.1.1 – Model Comparisons

The chosen models and approaches align with the project's goals of accuracy, scalability, and usability. Random Forest provides reliable crop recommendations, the rule-based approach ensures clarity in fertilizer guidance, and ResNet9 delivers effective disease detection with a balance of accuracy and efficiency.

7.2 RESULTS

1. System Performance:

- Disease Detection: Accurately identified crop diseases such as leaf blight and bacterial spots during testing, providing actionable insights for management.
- Soil Health Analysis: Successfully analyzed soil parameters, including pH, nutrient levels, and texture, to provide tailored recommendations for improved soil management.
- Real-Time Insights: Delivered rapid results on crop health and soil conditions, facilitating timely decision-making for farmers.

2. Key Features Delivered:

- Interactive Dashboard: The web application enables farmers to visualize crop health, soil quality, and disease alerts in an intuitive format.
- Disease Classification: Provides clear disease identification along with recommended remedial measures based on expert agricultural practices.
- Resource Optimization Recommendations: Offers actionable guidance for irrigation, fertilization, and pest management to enhance resource efficiency.

3. Impact on Farming Practices:

- Improved Decision-Making: Farmers gain deeper insights into crop health and soil quality, enabling more informed choices to optimize yield.
- Streamlined Processes: Automated analysis reduces the time required for manual inspections and laboratory testing, saving effort and resources.
- Sustainability: Promotes eco-friendly farming by minimizing overuse of water, fertilizers, and pesticides through targeted recommendations.

4. Scalability and Accessibility:

- User-Friendly Interface: Designed to cater to farmers with varying technical

skills, ensuring broad usability.

- Broad Applicability: Scalable across different farm sizes and types, from smallholder farms to large agricultural enterprises.
- Offline Capability: Local storage options enable access to soil test results even without continuous internet connectivity.

5. Future Potential:

- Customizable Models: The modular design of the system allows for the addition of new crops, diseases, and environmental parameters based on user requirements.
- Geographic Expansion: The system is adaptable for use in diverse regions, providing opportunities for global deployment.

This results section emphasizes practical achievements and tangible outcomes of the project, aligning closely with the system's capabilities and objectives.

CHAPTER-8

FUTURE SCOPE

1. Advanced Disease Severity Analysis

- Enhance the system to quantify disease severity, providing prioritized treatment recommendations.

2. Multi-Language Support

- Introduce multi-language interfaces to make the system accessible to farmers worldwide.

3. Mobile Application Development

- Develop a mobile application for easy access to real-time crop health data and alerts.

4. Blockchain Integration

- Implement blockchain for secure and transparent data sharing within the agricultural supply chain.

5. Real-Time Resource Management

- Enable automated systems for irrigation and fertilization using data-driven insights.

6. Integration with Drones

- Utilize drones for on-demand, high-resolution imagery to supplement satellite data.

7. Collaboration with Agricultural Platforms

- Partner with government and private organizations for subsidies, market insights, and crop pricing forecasts.

8. Scalability Across Regions

- Adapt the system for diverse climates, soil types, and crop varieties to extend its global applicability.

CHAPTER-9

CONCLUSION

In conclusion, this project effectively combines advanced technologies such as satellite imagery, machine learning, and web-based solutions to create a robust system for crop health monitoring and disease detection. By overcoming the challenges of traditional agricultural methods, the proposed system offers real-time insights, disease detection, and actionable recommendations, empowering farmers to optimize resources and enhance crop yields.

The system's use of Convolutional Neural Networks (CNNs) for disease detection and rule-based approaches for fertilizer recommendations, along with its integration of Google Earth Engine for satellite data, ensures accuracy and efficiency. A user-friendly web interface and local SQLite database storage make the system accessible to farmers of all scales, while the inclusion of Kaggle datasets enhances its ability to deliver precise, data-driven solutions.

By enabling early detection and intervention, the project reduces risks associated with crop diseases and promotes sustainable farming practices. It addresses critical agricultural challenges and demonstrates the potential of technology to transform traditional farming. This system sets the stage for future advancements in precision agriculture, contributing to increased productivity, reduced waste, and greater food security on a global scale.

REFERENCES

- [1] Kathiresan, G., & Karthik, R. (2021). *Enhancing Rice Crop Health: A Lightweight CNN-Based Disease Detection System with Mobile Application Integration*. Multimedia Tools and Applications. Retrieved from <https://www.springer.com>
- [2] Vadivel, T., & Suguna, R. (2022). *Revolutionizing Agriculture with Artificial Intelligence: Plant Disease Detection Methods, Applications, and Their Limitations*. Frontiers in Plant Science. Retrieved from <https://www.frontiersin.org>
- [3] MDPI Team. (2022). *Machine Learning for Detection and Prediction of Crop Diseases*. MDPI. Retrieved from <https://www.mdpi.com>
- [4] Bi, C., Wang, J., & Duan, Y. (2022). *Computer Vision for Plant Disease Recognition*. The Botanical Review. Retrieved from <https://www.botanicalreview.com>
- [5] Remote Sensing Team. (2022). *Remote Sensing in Field Crop Monitoring: A Comprehensive Review of Data Fusion and Machine Learning Approaches*. MDPI. Retrieved from <https://www.mdpi.com>
- [6] Rajvanshi, A., & Chin, S. W. (2023). *Early Detection of Crop Diseases Using CNN Classification*. NHSJS. Retrieved from <https://nhsjs.com>
- [7] Agronomy Team. (2023). *Drones in Plant Disease Assessment*. Agronomy Journal. Retrieved from <https://www.agronomy.org>
- [8] Nature Team. (2023). *Image-Based Crop Disease Detection Using Federated Learning*. Nature. Retrieved from <https://www.nature.com>
- [9] Remote Sensing Team. (2023). *Recent Advances in Crop Disease Detection Using UAV and Deep Learning Techniques*. Remote Sensing. Retrieved from <https://www.mdpi.com>
- [10] Environmental Monitoring Team. (2023). *Revolutionizing Crop Disease Detection with Computational Deep Learning*. Retrieved from <https://www.springer.com>
-

- [11] Plant Methods Team. (2023). *Plant Diseases and Pests Detection Based on Deep Learning: A Review*. Retrieved from <https://plantmethods.biomedcentral.com>
- [12] Springer Team. (2024). *Integrative Approaches in Modern Agriculture*. Retrieved from <https://www.springer.com>
- [13] Jariwala, S., & Jain, R. (2024). *A Convolutional Neural Network Approach to Robust Crop Health Monitoring*. ResearchGate. Retrieved from <https://www.techrxiv.org>
- [14] MDPI Team. (2024). *Crop Monitoring Using Satellite/UAV Data Fusion and Machine Learning*. MDPI. Retrieved from <https://www.mdpi.com>
- [15] Springer Team. (2024). *Transforming Agriculture with Machine Learning, Deep Learning, and IoT*. Springer. Retrieved from <https://www.springer.com>
- [16] IEEE Team. (2024). *Precision Agriculture with AI: Data Fusion and Advanced Analytics*. IEEE. Retrieved from <https://www.ieee.org>
- [17] Springer Team. (2024). *Automated Pest Detection with Deep Neural Networks on the Edge for Precision Agriculture*. Springer. Retrieved from <https://www.springer.com>
- [18] Springer Team. (2024). *Satellite-Based Crop Health Monitoring: Integrating Remote Sensing and Machine Learning*. Springer. Retrieved from <https://www.springer.com>
- [19] Springer Team. (2024). *Deep Learning for Image-Based Plant Disease Detection: A Review*. Springer. Retrieved from <https://www.springer.com>
- Springer Team. (2024). *IoT and Machine Learning in Agriculture: A Survey*

Research Paper Accepted for Publication - DOI for Research Paper - Indexing
Google Scholar, Academia, Research Gate



IJSREM Journal <editor@ijsrem.com>
to me

Mon, Dec 30, 2024, 9:01AM (4 days ago)



Dear Author,

We are pleased to inform you that your research Paper "CropSense: An Integrated Web App to Simulate a ML/DL-Based Decision Support System for Precision Farming and Agriculture" for publication in **"International Journal of Scientific Research in Engineering and Management (IJSREM)"** on Volume 08 Issue 12 December - 2024. We are delighted to see your commitment and hard work to share your research is being recognized.

Your Research Paper ID : [IJSREM40325](#)

Please kindly pay any one of publication charges to publish your research paper work.

With DOI: ₹ 1000/- INR





Open Access
INTERNATIONAL JOURNAL

V10I6-1514: Registration fee received

Dear Author

This is to inform you that you have successfully registered for the publication of your document ID: V10I6-1514.

Please track your paper for any correction or addition in the paper or author details. Once the paper is published, no changes will be made to the paper.

Your paper will now be queued for editing and formatting changes, once completed it will be published immediately on the portal.

In case of any query, [reply to this email](#) or call our counselors at [+91-8195072273](#)

Track Status	
Paper ID	V10I6-1514 (v10i6-1514)
Title	CropSense: An Integrated Web App to Simulate a ML/DL-Based Decision Support System for Precision Farming and Agriculture
Author(s)	Adrian Mathew Aloysius, Dave Joseph Pinto, Narjit Leishangthem, Mohammed Affan, Mr. Shyam Dev R S, Dr. D Roja Ramani
Status	Accepted
Tracking ID	v10i6-1514-adr (Track Now)



It is to inform you that this email is being sent only to the first author of this paper. We are assuming that you are conveying this message to other authors on your own. If you want us to send this email to all the authors from our side, please revert to this email.

Follow us on 

M

To Mr. Rohit Kumar

₹1,540

IJARIIT

Pay again

Completed

29 Dec 2024, 6:48 pm

HDFC Bank 2101

UPI transaction ID

473057265068

To: Mr. Rohit Kumar

9592898200@idfcfirst

From: ADRIAN MATHEW ALOYSIUS (HDFC Bank)

adrianalloysius02@okaxis

Google transaction ID

CiCAgliNp_zGDQ

POWERED BY





DEPARTMENT OF CSE, NHCE

100

